SKALA

AI Campus 데이터분석 및 AIOps

4. 머신러닝 및 딥러닝 이해
Part B – Deep Learning

2026.08

배기주 / SK AX



본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX



Part B – Deep Learning

1. 회귀분석 재해석
2. 모델 최적화
3. 데이터 구조




That's Bananas!
Highest resistant starch
Sweeter
Super sweet
Underripe
Barely ripe
Ripe
Very ripe
Overripe
A source of prebiotics
Higher antioxidant levels
Greatest vitamin + mineral loss
Highest antioxidant levels







Rule by Human

길다
노란색
약간 휘었다

하얀색
납작하다
둥글다

바나나

Rule-based, Statistics





바나나

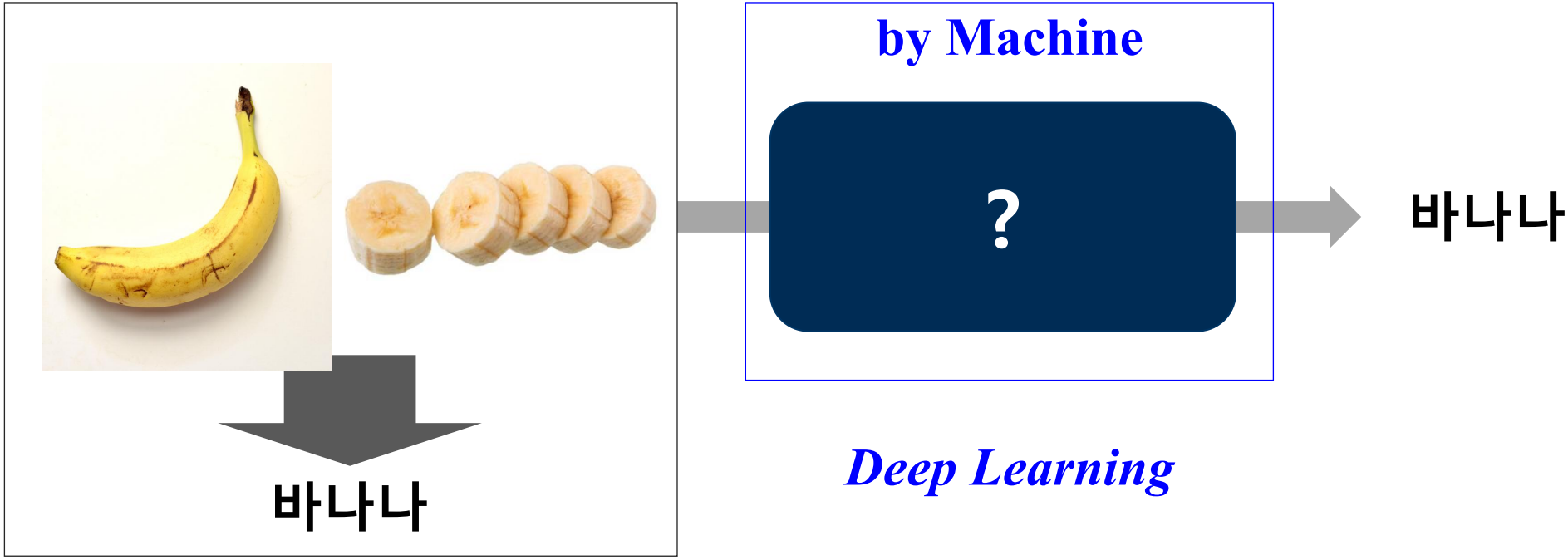
by Machine

길다
노란색
약간 휘었다

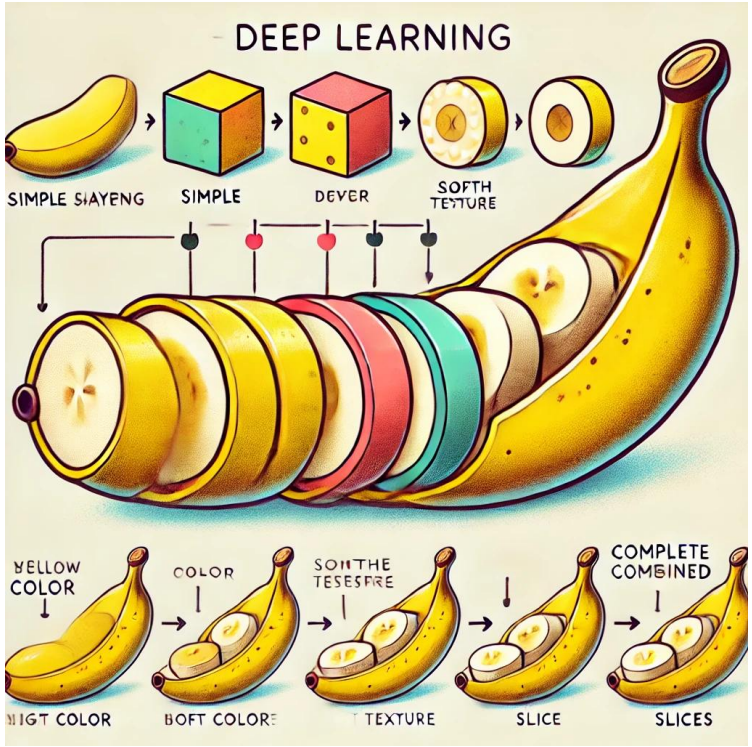
하얀색
납작하다
둥글다

바나나나

Machine Learning


by Machine
?
바나나
바나나
Deep Learning

바나나와 함께 하는 딥러닝



바나나와 함께 하는 딥러닝

개념: 딥러닝은 여러 단계로 이루어진 과정이라고 생각할 수 있어요. 마치 바나나를 인식할 때 여러 단계로 알아보는 것과 비슷해요. 바나나 껍질을 벗기거나, 조각을 내서 자세히 보는 것처럼, 딥러닝도 여러 층을 통해 더 깊이 알아가는 거예요.

단계별 설명:

1. **첫 번째 층 (기본 모양):** 딥러닝의 첫 번째 층은 가장 기본적인 모양을 살펴봐요. 이 층에서는 물체가 길고 휘어져 있다는 걸 알아요. 바나나의 바깥 모양을 본 것과 비슷해요.
2. **두 번째 층 (색깔과 질감):** 다음 층에서는 더 깊이 들어가서 물체의 색깔과 질감을 살펴봐요. 이 층은 노란색이고 껍질이 매끄럽다는 걸 알아차려요. 이제는 단순히 길고 휘어진 게 아니라 노란색인 물체로 더 정확히 알 수 있어요.
3. **세 번째 층 (안쪽의 세부사항):** 또 다른 층에서는 바나나를 잘랐을 때 보이는 안쪽을 살펴봐요. 이 층은 바나나 안쪽이 밝은 색이고, 부드럽고 동그란 조각이라는 걸 알아요. 이렇게 한 층 한 층 지나가면서 바나나에 대한 세부사항을 추가로 알게 돼요.
4. **최종 이해:** 모든 층을 거치면서 딥러닝은 각 층에서 얻은 정보를 합쳐서 "이건 분명히 바나나야!"라고 확신해요.

비유: 이걸 여러 개의 안경을 끼고 보는 것처럼 생각해볼 수 있어요. 첫 번째 안경은 큰 모양을 볼 수 있게 해주고, 다음 안경은 색깔을 볼 수 있게 해주고, 마지막 안경은 안쪽을 보여줘요. 이렇게 여러 개의 안경(층)이 모여서 바나나를 완전히 알아볼 수 있게 해주듯, 딥러닝의 층도 함께 일하면서 사물을 정확히 인식하게 도와주는 거예요.

이런 층층이 쌓인 딥러닝 과정은 컴퓨터가 사물을 더욱 잘 인식할 수 있게 해줘요. 마치 바나나를 한 껍질씩 벗기며 속을 알아보는 것처럼요!

단계별로...

- 단계별로 정확하게 특징을 파악하고 있는가?
- 단계별로 모델이 예측을 잘 하고 있는가?
- 단계별로 실제값과 예측값의 오차는 얼마나 되는가?

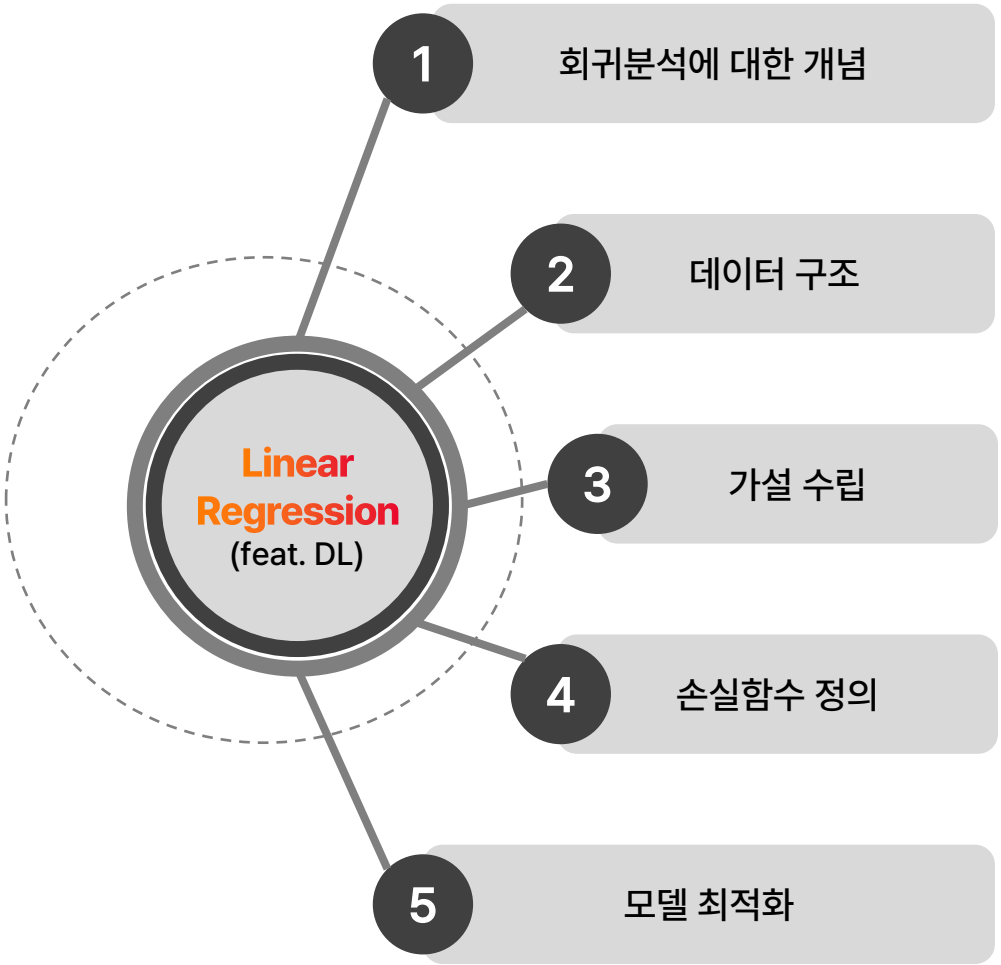
➢ Learning, Weights & Bias

➢ Minimizing Error

➢ MSE, Least Squares Method

Linear Regression 재해석

Key Point



- **회귀분석에 대한 개념 이해**
  - 회귀분석의 정의 및 활용 목적
  - 딥러닝 적용 위해 필요한 추가 절차
- **Data Definition, 데이터 구조에 대한 이해**
  - 딥러닝에 적합한 데이터 형태
  - Tensor Shape 이해
- **수립한 가설 기반으로 모델 개발**
  - 가설 수립 방법의 이해
  - 선형모델 또는 딥러닝 신경망 네트워크 아키텍처 구조 적용
- **손실함수 정의 및 계산**
  - 모델의 적합 여부를 평가하기 위한 기준의 손실 함수 정의/적용
  - 목적별 손실함수 이해
- **최적화 개념 및 방법**
  - Gradient Descent 개념 이해
  - 최적화 종류 및 종류별 장단점 이해/적용

AI Campus 데이터분석 및 AIOps > Part B – Deep Learning

1. 회귀분석 재해석

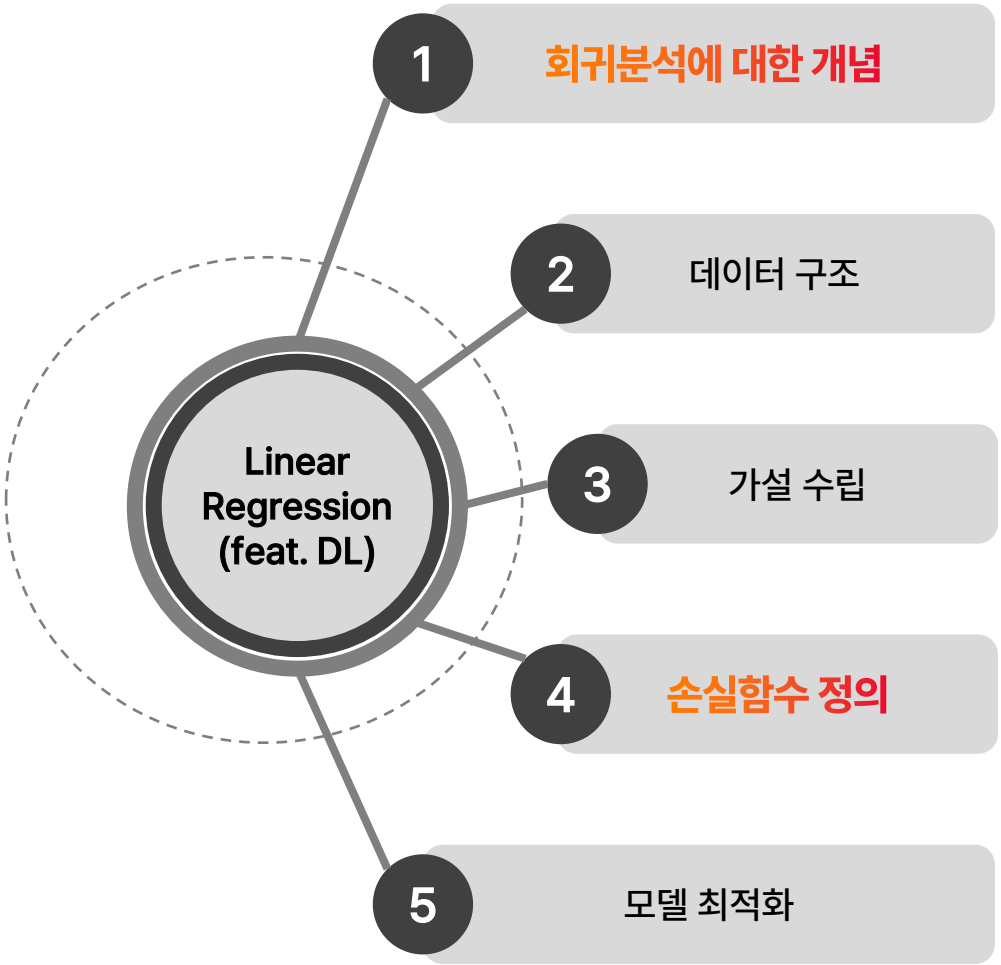
- 회귀분석의 정의 및 활용 목적
- 딥러닝 적용 위해 필요한 추가 절차

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

Linear Regression 재해석

Key Point



- **회귀분석에 대한 개념 이해**
  - 회귀분석의 정의 및 활용 목적
  - 딥러닝 적용 위해 필요한 추가 절차
- **Data Definition, 데이터 구조에 대한 이해**
  - 딥러닝에 적합한 데이터 형태
  - Tensor Shape 이해
- **수립한 가설 기반으로 모델 개발**
  - 가설 수립 방법의 이해
  - 선형모델 또는 딥러닝 신경망 네트워크 아키텍처 구조 적용
- **손실함수 정의 및 계산**
  - 모델의 적합 여부를 평가하기 위한 기준의 손실 함수 정의/적용
  - 목적별 손실함수 이해
- **최적화 개념 및 방법**
  - Gradient Descent 개념 이해
  - 최적화 종류 및 종류별 장단점 이해/적용

1. 회귀분석에 대한 개념

회귀방정식 이란?

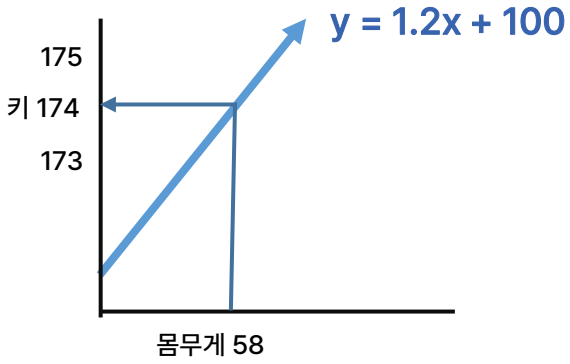
목적

- 입력 받은 몸무게로 "키(Y)" 예측

데이터 수집

몸무게(X) weight	키(Y) Height
58	174
59	175
60	176
….	….

선형 회귀식



【 구하고 싶은 것은? 】

- y = ax + b
- **기울기(a)와 절편(b) ➔ 회귀계수**

회귀계수를 구하기 위한 접근 방안

(1) a는 현재 모르는 값이나, 궁극적으로 구해야 할 값임.
일단 임의의 값을 정하여 대입해 보자. (a=1.2, b=100)

(2) 대입한 임의의 값으로 몸무게(X)=58 활용하여 키(Y) 값을 계산해 보자.
y = 1.2 * 58 + 100 = 169.6

(3) 키(Y) 값의 실제값과 예측값을 비교 하면…
실제값 174 – 예측값 169.6 = 차이 4.4

(4) 실제값과 예측값의 차이가 0이 되도록 반복하면,
0에 가까워지는 a와 b를 구할 수 있음

1. 회귀분석에 대한 개념

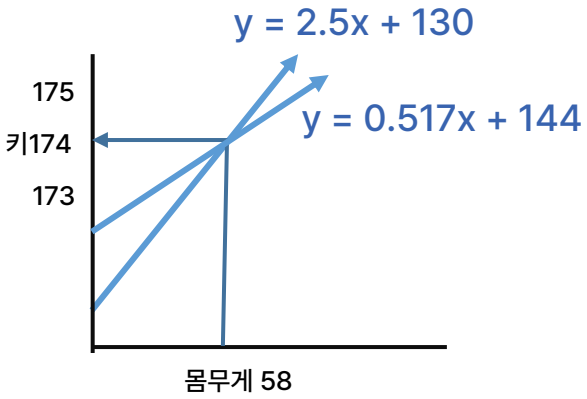
선형모델에서 알고 싶은 것은?

몸무게(X) weight	키(Y) Height
58	174
59	175
60	176
....	....

가설, Hypothesis

$$y = Wx + b$$

- 회귀계수
- 가중치
- 기울기
- 절편
- Bias



【 선형모델에서 알고 싶은 것은? 】

알고 있는 것 : x, y

- 데이터 수집을 통해 DB에 저장된 값
- x : 독립변수(Feature)는 모델을 만들기 위해 입력되는 값
- y : 종속변수(Target). 결과값으로 모델 예측값과 비교

알고 싶은 것 : W, b

- 가중치(W)에 따라 다양한 기울기 선이 존재
- **기울기(W)와 절편(b)은 파라미터라 하고, 알고 있는 데이터(x, y)를 학습시켜 결정됨**

- 딥러닝에서는 최적의 파라미터를 계산하는 방법 중 대표적인 방법으로 경사하강법(Gradient Descent)

2. 파라미터 구하는 과정

선형 회귀분석인 경우...

- 파라미터 : 기울기(W)와 절편(b)
- 알고 있는 데이터(x, y)를 학습시켜 기울기(W)와 절편(b) 결정

Step 1 데이터 정의 (x, y)

몸무게 x	키 y
58	174
59	175
60	176
….	…

Step 2 선형 모델 정의 (가설수립)

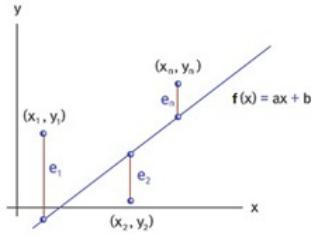
$$y = Wx + b$$

- 회귀계수
- 가중치
- 기울기
- 절편
- Bias

Step 3 오차 평가, 손실 함수 정의 (MSE : Mean Squared Error)

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

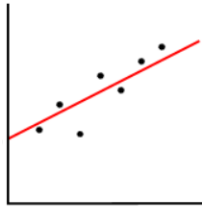
Step 4 오차 최소화 (Least Squares Method)



$$b = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$$

Step 5 파라미터 계산 (W, b)

$$y = 2.5x + 29$$



2. 파라미터 구하는 과정

딥러닝에 적용하면...!!!

- 경사하강법, Gradient Descent : 손실함수의 기울기(Gradient)를 낮은 값 쪽으로 이동(하강)시켜 근사값을 찾는 최적화 알고리즘, 손실함수가 최소가 되는 최적의 파라미터를 찾는 과정
- 이를 통해 회귀식 파라미터인 기울기와 절편의 최적의 해를 찾음

Step 1 데이터 정의 (Feature, Target)

몸무게	키
Feature	Target
58	174
59	175
60	176
….	…

데이터는 준비되었고

Step 2 선형 모델 정의 (가설수립)

$$y = Wx + b$$

- 회귀계수
- 가중치
- 기울기
- 절편
- Bias

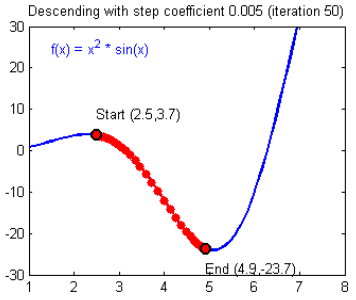
모델 구조 설정했고 (W, b 초기화)

Step 3 오차 평가, 손실 함수 정의 (MSE : Mean Squared Error)

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

모델을 평가할 기준도 마련했고

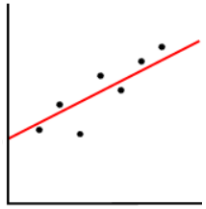
Step 4 오차 최소화, **경사하강법**



손실함수가 최소가 되는 파라미터를 찾아서

Step 5 파라미터 계산 (W, b)

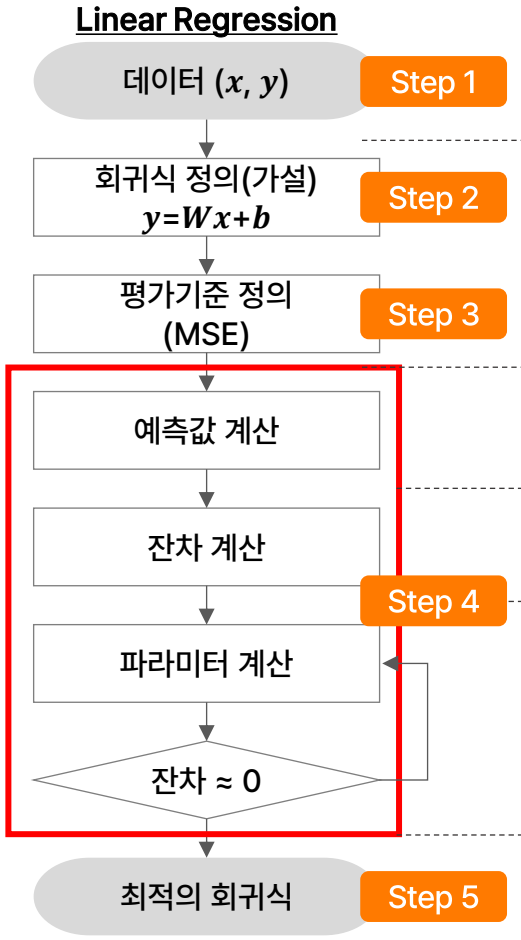
$$y = 2.5x + 29$$



모델 학습 통해 최적화된 파라미터 업데이트

3. 회귀식을 구하는 과정

선형 회귀분석인 경우...



- **데이터 수집/저장**
- **학습 데이터셋 정의**

- **문제 정의 따라 Regression or Classification 정의**
  - Regression : $y=Wx+b$
  - 평가 기준 : MSE (실제값 – 예측값), RMSE ($\sqrt{\text{MSE}}$)

- **학습 모델 통해 예측값 계산**
  - 계산식 (Wx+b)에 실제값 X을 입력하여 계산되는 예측값

- **잔차(오차) 계산**
  - 계산식을 통해 계산된 예측값과 실제값의 차이 계산

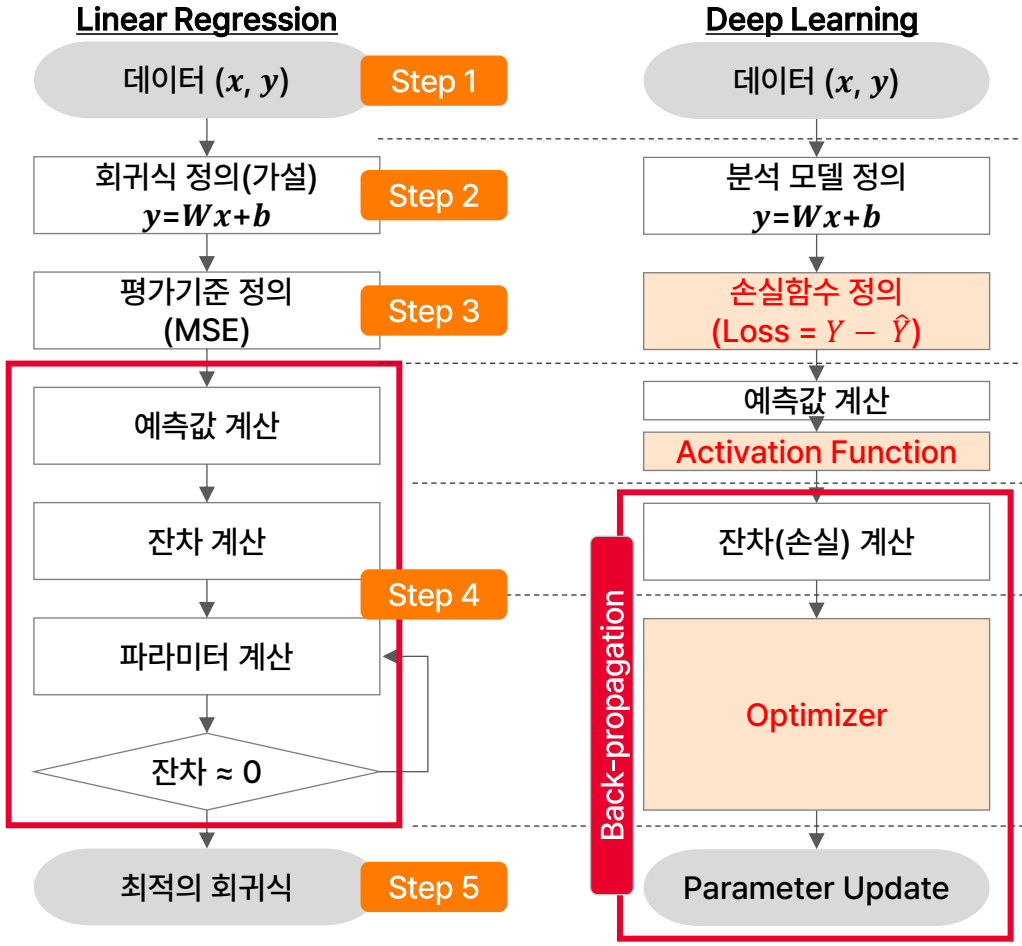
- **파라미터(W, b) 업데이트**
  - 파라미터를 업데이트 하는 규칙(최소자승법)을 정의하고, 이를 통해 W와 b 값 조정
  - 해당 과정을 반복 횟수만큼 진행하여 오차가 최소(잔차 ≈ 0)가 되는 최적의 파라미터 계산

- **최적의 회귀식 추정**
  - 오차가 최소가 되는 최적의 파라미터로 최적의 회귀식 정의

3. 회귀식을 구하는 과정

Key Point

Statistics vs Deep Learning



- 데이터 수집/저장
- 학습 데이터셋 정의 (x →Feature, y →Target)

- **문제 정의 따라 Regression or Classification 정의**
  - 평가 기준 [ML] MSE (실제값 – 예측값), RMSE ($\sqrt{MSE}$)
    [DL] 손실함수, 비용함수 Cross Entropy *Classification*
  - ← **손실(오차)를 수치적으로 측정해 Backpropagation에 필요 정보 제공**

- **학습 모델 통해 예측값 계산**
  - 예측값에 Activation Function을 적용하여 **비선형성 추가** (ReLU)

- **잔차(오차, 손실, 비용함수) 계산**
  - **잔차를 줄이기 위해** Back-propagation **진행**

- **파라미터(W, b) 업데이트**
  - 파라미터를 업데이트 하는 규칙을 정의하고, 이를 통해 W와 b 값 조정
  - 오차가 최소(잔차 ≈ 0)가 되는 최적의 파라미터 계산 (**경사하강법**)
  - Back-propagation**은 손실이 가장 빠르게 줄어드는 방향으로, 최적화 알고리즘으로 가중치와 파라미터를 어떻게 조정할지 결정**

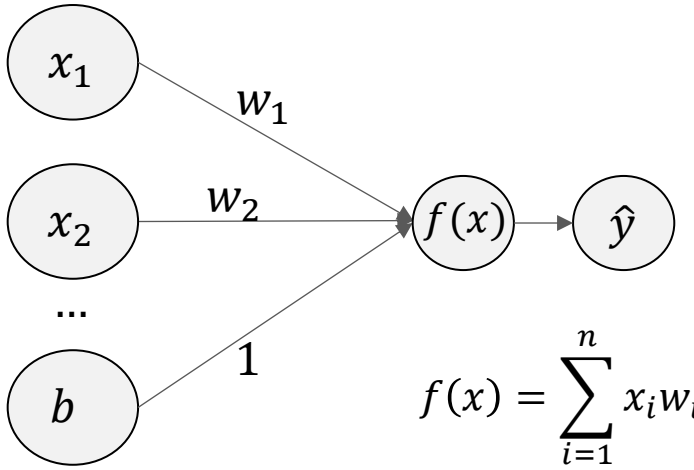
- **최적의 회귀식 추정**
  - 오차가 최소가 되는 최적의 파라미터로 최적의 회귀식 정의
  - **계산된 조정값을 적용하여 가중치 업데이트 (반복학습,** Epoch)

3. 회귀식을 구하는 과정

Key Point

feat. Activation Function

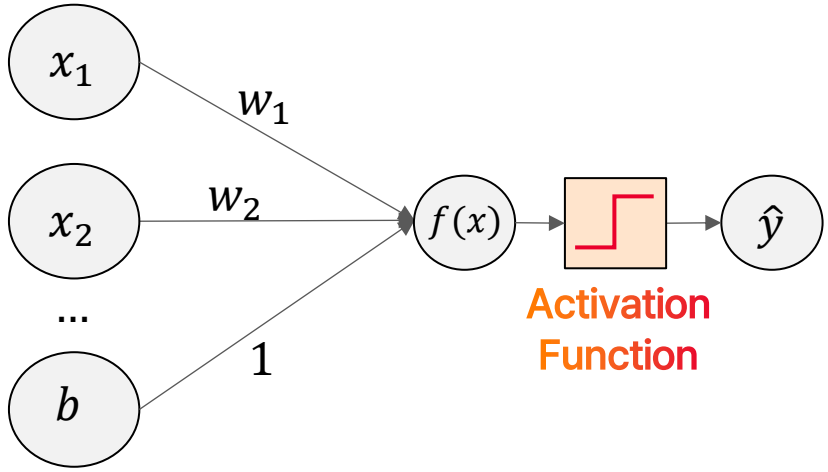
Linear Regression



$$f(x) = \sum_{i=1}^{n} x_i w_i + b$$

선형 회귀

Deep Learning

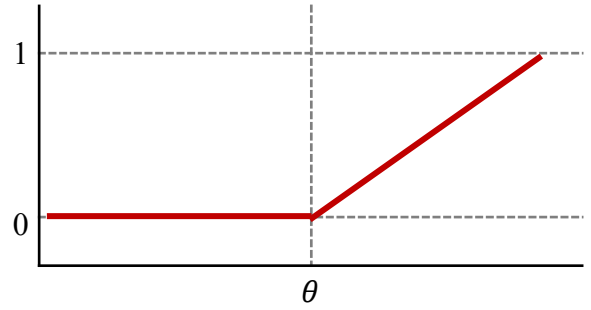


선형 회귀 + 활성함수

선형 회귀식을 비선형으로 만드는 활성함수를 가진 노드를 레이어로 쌓아 딥러닝 아키텍처로 구성

4. Activation Function

ReLU – Rectified Linear Function



- 정의 : $ReLU(x) = \max(0, x)$
- 출력 : $(0, \infty)$
- 특징 : 입력이 양수인 경우 그대로 출력, 음수인 경우 0을 출력

→ 비선형성을 추가하면서도 계산이 간단하고 효율적이라 많이 사용됨

입력된 신호에서 중요한 부분은 그대로 전달,
필요 없는 음의 값은 0으로 만들어서
효율적으로 학습할 수 있도록

신경망이 다양한 특성과 패턴을 인식하고
정확하게 예측/분류 하는데 필수

예시: 사과와 오렌지 구분하기

1. 문제 설정:

- 사과와 오렌지를 구분하기 위해 우리는 크기(예: 지름)와 색상(예: 빨강, 주황 정도)을 특성으로 사용한다고 가정합시다. 이 특성들은 각 과일을 설명하는 데이터 포인트로 생각할 수 있습니다.

2. 데이터 입력과 ReLU의 적용:

- 딥러닝 모델에 크기와 색상을 입력하면, 입력값은 첫 번째 층의 뉴런에 전달됩니다. 각 뉴런은 입력값에 가중치를 곱하고 편향을 더한 후, ReLU 함수를 적용합니다.
- 예를 들어, 어떤 뉴런이 사과와 오렌지를 구분하려고 할 때, 사과일 가능성이 높은 특성 조합(크기가 크고 빨간색)이 들어오면 ReLU는 이 값을 그대로 유지합니다. 반면, 오렌지일 가능성이 높은 특성 조합(크기가 작고 주황색)이 들어오면 그 값 중 일부는 0으로 줄여 버립니다.

3. ReLU의 역할:

- ReLU는 입력된 값에서 의미 있는 양성 신호(사과에 더 가까운 특성)를 그대로 전달하면서 의미 없는 음성 신호(오렌지에 가까운 특성)를 걸러내는 역할을 합니다.
- 예를 들어, 사과와 오렌지를 구분하는 데 크기가 중요한 경우, 크기가 0보다 작은 값이나 비슷한 경우에 ReLU는 이를 0으로 처리하여 신경망이 구분을 쉽게 하도록 돕습니다.

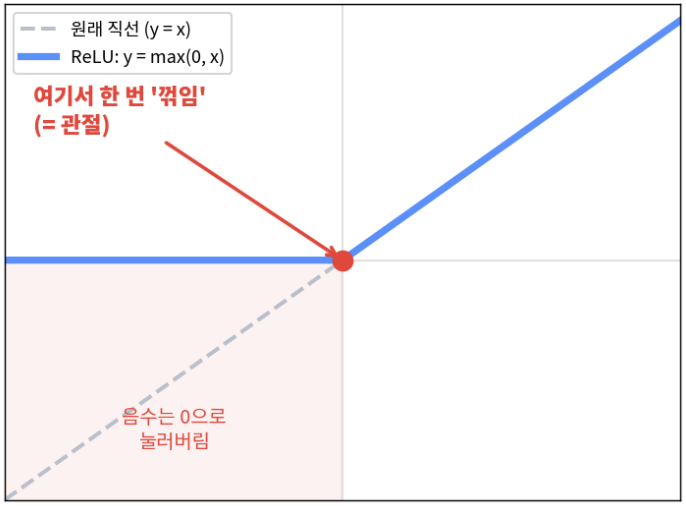
4. ReLU가 구분 경계에 미치는 영향:

- 신경망이 점차 훈련되면서, ReLU는 사과와 오렌지를 구분하는 결정 경계(비선형 경계)를 형성하는 데 기여합니다. 예를 들어, 뉴런이 사과를 대표하는 특성에 대해 큰 값을 유지하고, 오렌지에 대해선 0으로 만드는 방향으로 훈련된다면, 그 결과 사과와 오렌지를 구분하는 복잡한 비선형 경계가 형성됩니다.
- 이 경계는 단순한 직선이 아니라, ReLU의 비선형 특성 덕분에 더 복잡하고 정확하게 사과와 오렌지를 나눌 수 있는 모양으로 발전합니다.

[참고] ReLU, 쉽게 이해하기

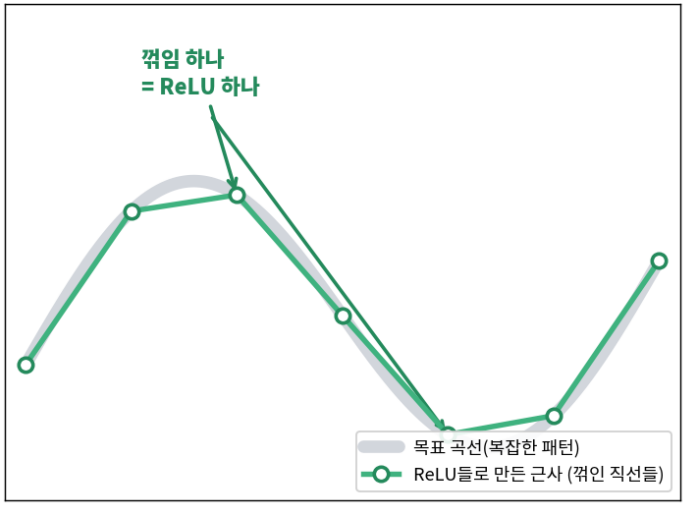
'음수 버리기'가 만드는 비선형성

① ReLU = 음수를 0으로 자른다



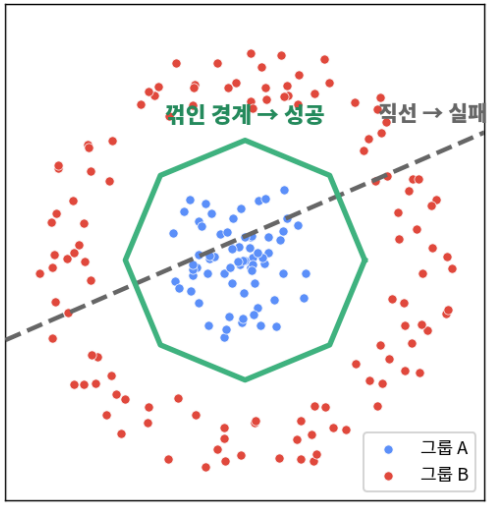
곧게 뻗던 직선이 0에서 '꺾인다' → 이 꺾임이 곧 비선형

② 꺾임을 여러 개 모으면 → 곡선도 흉내낸다



꺾임(ReLU)이 많을수록 더 복잡한 곡선 표현 가능

③ 그래서 직선으로론 못 나누던 걸 나눈다

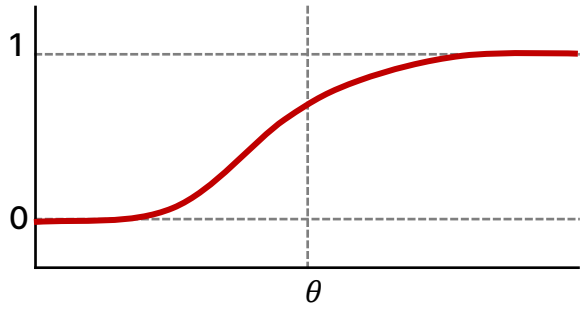


곧은 직선 하나로는 두 그룹을 못 가름 → 꺾인 경계라야 가능

4. Activation Function

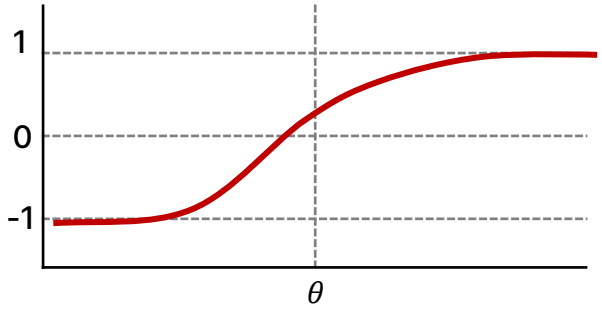
입력 데이터에 비선형성을 추가하여 복잡한 패턴과 관계를 학습할 수 있도록 하고 출력값의 범위를 제한하여 계산의 안정성 보장

【 Sigmoid 】



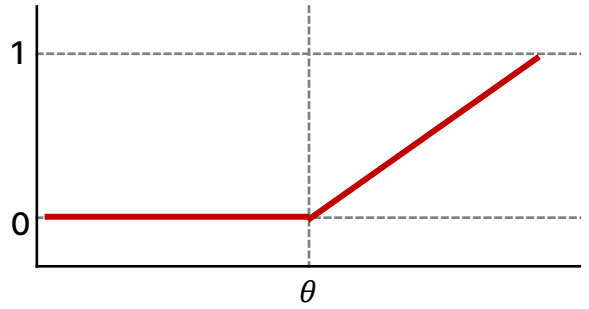
- 정의 : $\sigma(x) = \frac{1}{1+e^{-x}}$
- 출력 : (0, 1)
- 특징
  - 출력이 0과 1 사이로 제한되어 확률값을 출력하는데 유용
  - 입력값이 극단적인 경에도 출력값이 0 또는 1에 가까워지며, 이로 인해 Gradient vanishing 발생할 수 있음
- 활용
  - Binary Classification
  - Binary Probability (eg, Logistic Regression)

【 Tanh 】



- 정의 : $\tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$
- 출력 : (-1, 1)
- 특징
  - 출력이 -1과 1 사이로 제한
  - Sigmoid와 유사하지만 출력값이 0 중심으로 분포되어 Gradient vanishing 문제가 덜 발생
- 활용
  - **RNN, LSTM 에서 많이 사용**
  - 출력값이 양수와 음수로 나뉘는 문제

【 ReLU 】



- 정의 : $ReLU(x) = \max(0, x)$
- 출력 : $(0, \infty)$
- 특징
  - 입력이 양수인 경우 그대로 출력하고 음수인 경우 0을 출력
  - 계산이 간단하고 Gradient vanishing 문제 적음
  - 일부 뉴런이 죽는 현상이 발생할 수 있음
- 활용
  - **대부분의 신경망에서 기본적으로 많이 사용**
  - ReLU의 장단점을 보완한 다양한 Function으로 최적화 진행

AI Campus 데이터분석 및 AIOps > Part B – Deep Learning

2. 모델 최적화

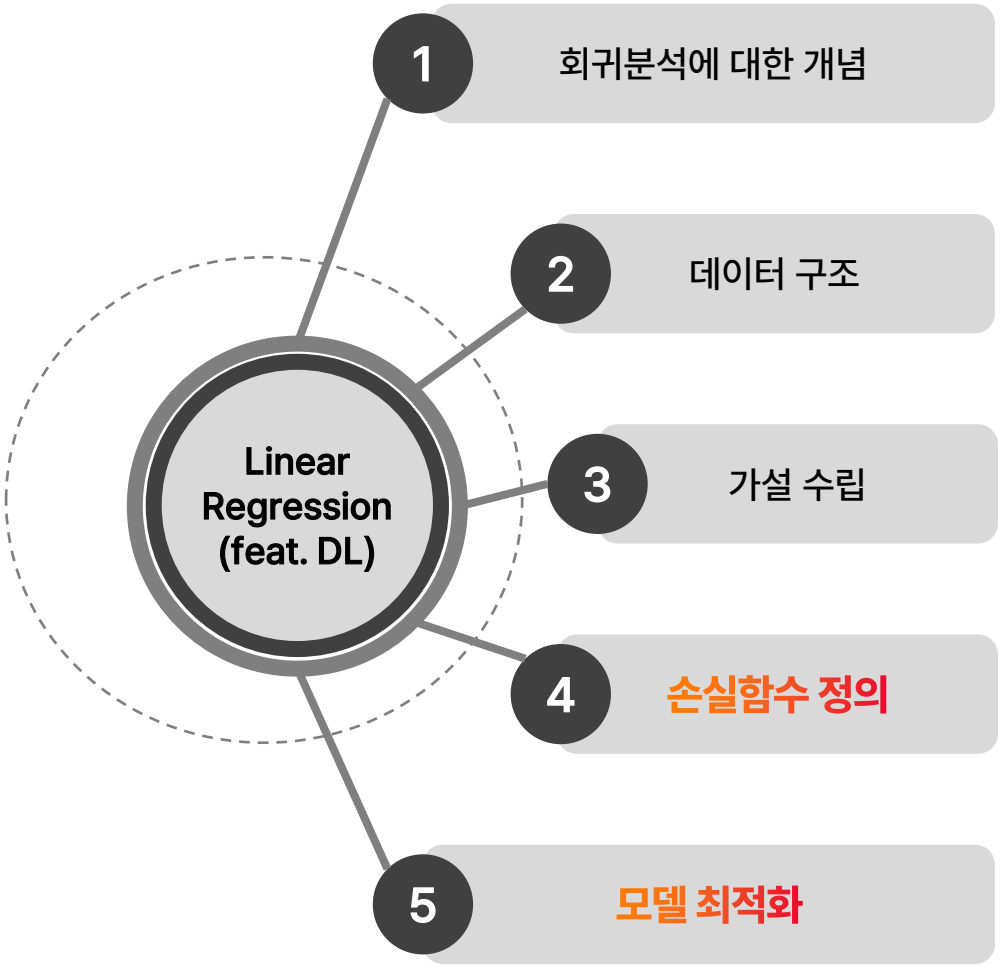
- Gradient Descent 개념 이해
- Forward & Backward Propagation

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

Linear Regression 재해석

Key Point



- **회귀분석에 대한 개념 이해**
  - 회귀분석의 정의 및 활용 목적
  - 딥러닝 적용 위해 필요한 추가 절차
- **Data Definition, 데이터 구조에 대한 이해**
  - 딥러닝에 적합한 데이터 형태
  - Tensor Shape 이해
- **수립한 가설 기반으로 모델 개발**
  - 가설 수립 방법의 이해
  - 선형모델 또는 딥러닝 신경망 네트워크 아키텍처 구조 적용
- **손실함수 정의 및 계산**
  - 모델의 적합 여부를 평가하기 위한 기준의 손실 함수 정의/적용
  - 목적별 손실함수 이해
- **최적화 개념 및 방법**
  - Gradient Descent 개념 이해
  - 최적화 종류 및 종류별 장단점 이해/적용

1. Approach

용어의 이해

최적화, 最適化, Optimization

- **주어진 조건이나 제약 내에서 최상의 결과를 얻도록 과정이나 방법을 조정하거나 개선하는 과정**
- 자원, 시간, 비용 등을 최소화 하거나 효율성, 효과성 등을 최대화 하는 것이 주된 목표
- (예) 경제학에서의 자원 배분, 공학에서의 설계 최적화, 물류에서의 경로 최적화 등

머신러닝 모델링에서의 최적화

- 모델의 학습 과정에서 **성능 지표를 개선**하기 위해 파라미터를 조정하고, 모델을 조정하여 최상의 성능을 얻는 과정
- 주요 목표 :
  - ❑ **성능 향상** : 모델이 정확하게 예측하고 좋은 성능을 발휘하도록 조정
  - ❑ **일반화 향상** : 새로운 데이터에 대해 잘 작동할 수 있도록 모델 개선
  - ❑ **효율성 증가** : 학습 과정의 속도를 높이고, 계산 자원을 효율적으로 사용

1. Approach

최적화 접근

최적화, 最適化, Optimization

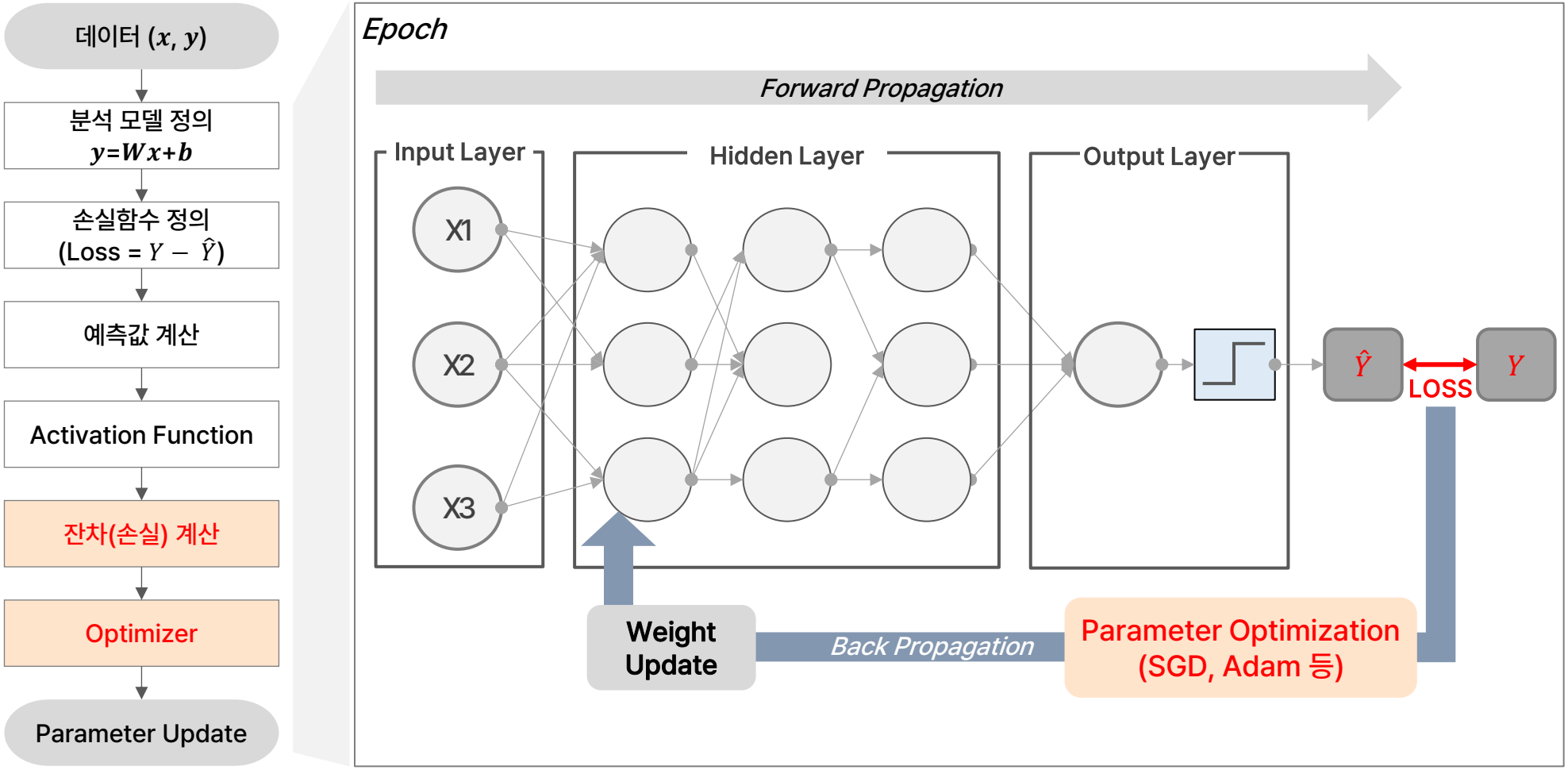
- **주어진 조건이나 제약 내에서 최상의 결과를 얻도록 과정이나 방법을 조정하거나 개선하는 과정**
- 자원, 시간, 비용 등을 최소화 하거나 효율성, 효과성 등을 최대화 하는 것이 주된 목표
- (예) 경제학에서의 자원 배분, 공학에서의 설계 최적화, 물류에서의 경로 최적화 등

딥러닝 모델에서의 수학적 접근

- 특정 함수의 값을 최소화 또는 최대화 시키는 **최적의 수식(=최적의 파라미터)** 조합을 찾는 문제
- **실제값과 예측값의 차이가 최소(Minimization)가 되는 방향**으로 최적화 수행
  Loss Function

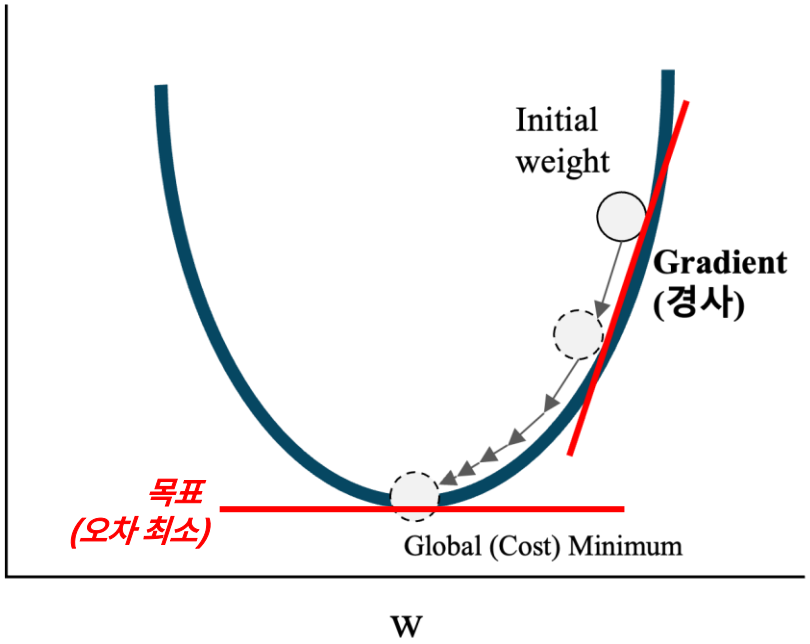
2. DL 최적화

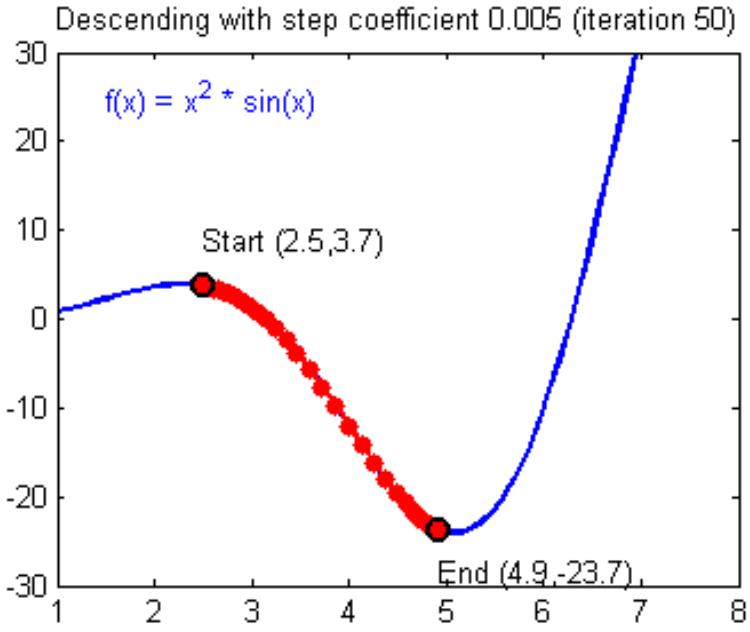
DL 모델 최적화는 Loss Function을 최소화 하는 파라미터를 구하는 과정으로 경사하강법을 주로 사용함



3. Gradient Descent

- Optimizer 초기 버전. Gradient Descent, 경사하강법
- 함수의 기울기(= 경사, Gradient)를 이용하여 X값을 어디로 옮겼을 때 함수가 최소값을 가지는지 알아내는 방법



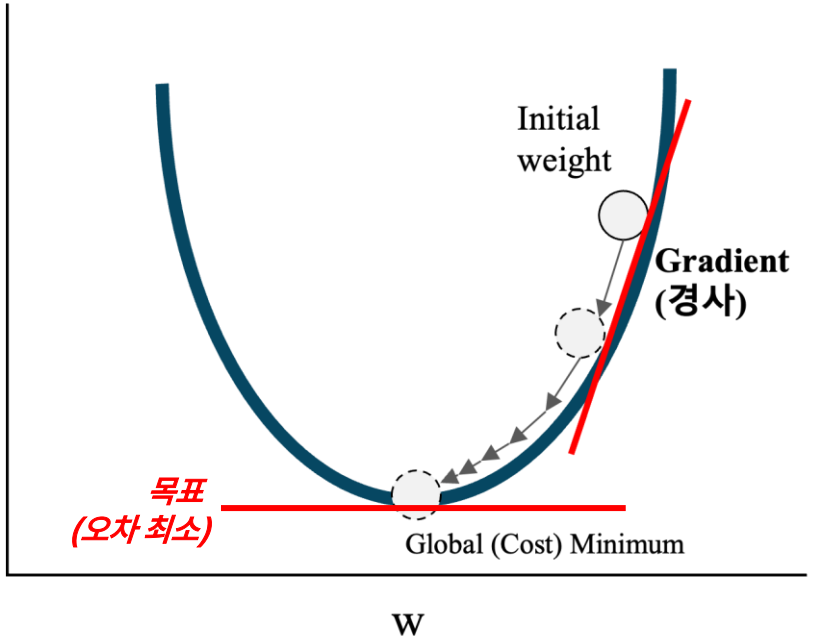


4. Learning Rate

Key Point

가중치가 얼마나 변할지, 업데이트 할지를 조정하는 값으로 목표 지점(Global Minimum)에 가까워질 수 있도록 최적화 필요

→ *Hyper-parameter Tuning*



- 학습률, Learning Rate
  - 0과 1 사이의 값을 가지며, 사람이 직접 설정 (Hyper-parameter)
  - 작을수록 : 촘촘하게 파라미터를 변화시킴, 학습시간 오래 걸림
  - 클수록 : 손실함수 그래프 자체를 벗어나는 경우가 발생할 수 있음 (Over-shooting)
- 적정 학습률
  - 10-5승의 비율로 시작, Hyper-parameter Tuning 통해 최적화
  - 학습 초기에는 큰 스텝으로 빠르게 탐색하다가, 최적화 포인트 (전역 최소점, Global Minimum)와 가까워질수록 스텝을 작게 조정하여 정밀하게 탐색
- 기울기($\frac{\partial E}{\partial W_t}$)
  - 손실함수(E)의 가중치(W_t)에 대한 변화율
  - 현재 가중치에서 손실함수가 얼마나 빠르게 변하고 있는지 (학습)방향과 크기를 포함한 정보

경사하강법의 Learning Rate

새로운 가중치 $*W_t$ = 현재 가중치 W_t − 학습률 η * 기울기 $\frac{\partial E}{\partial W_t}$

현재 가중치(W_t)에서
학습률(η)에 손실함수의 가중치에 대한 기울기($\frac{\partial E}{\partial W_t}$) 를 사용해
새로운 가중치($*W_t$)를 계산

[참고] Optimizer & LR 쉽게 이해하기

1. 산에서 내려오는 길 찾기
   - **Gradient Descent = 산길 내려가기**
   - 우리가 "산 꼭대기 (초기 가중치)"에서 출발해서 "산 아래 (최적점)"로 내려가는 상황
   - 이 때 "경사면(기울기)" 보고 방향을 결정하고 내려가는 것이 Gradient Descent

2. 한 걸음에 내려가는 거리
   - **Learning Rate = 한 걸음에 내려가는 거리**
   - 산을 내려갈 때 한번에 얼마나 큰 걸음으로 내려갈지 결정하는 것
   - Learning Rate 작음 : 아주 조심스럽게 한 발씩 내려감 → 너무 오래 걸림 (아장아장)
   - Learning Rate 큼 : 두세 발자국씩 뛰어서 내려감 → 너무 멀리 뛰어 길을 벗어남 (최적점 지나칠 수 있음, Over-shooting)

!! 공식 풀이

새로운 가중치 현재 가중치

$$^{*}W_t = W_t - \eta * \frac{\partial E}{\partial W_t}$$

학습률 기울기

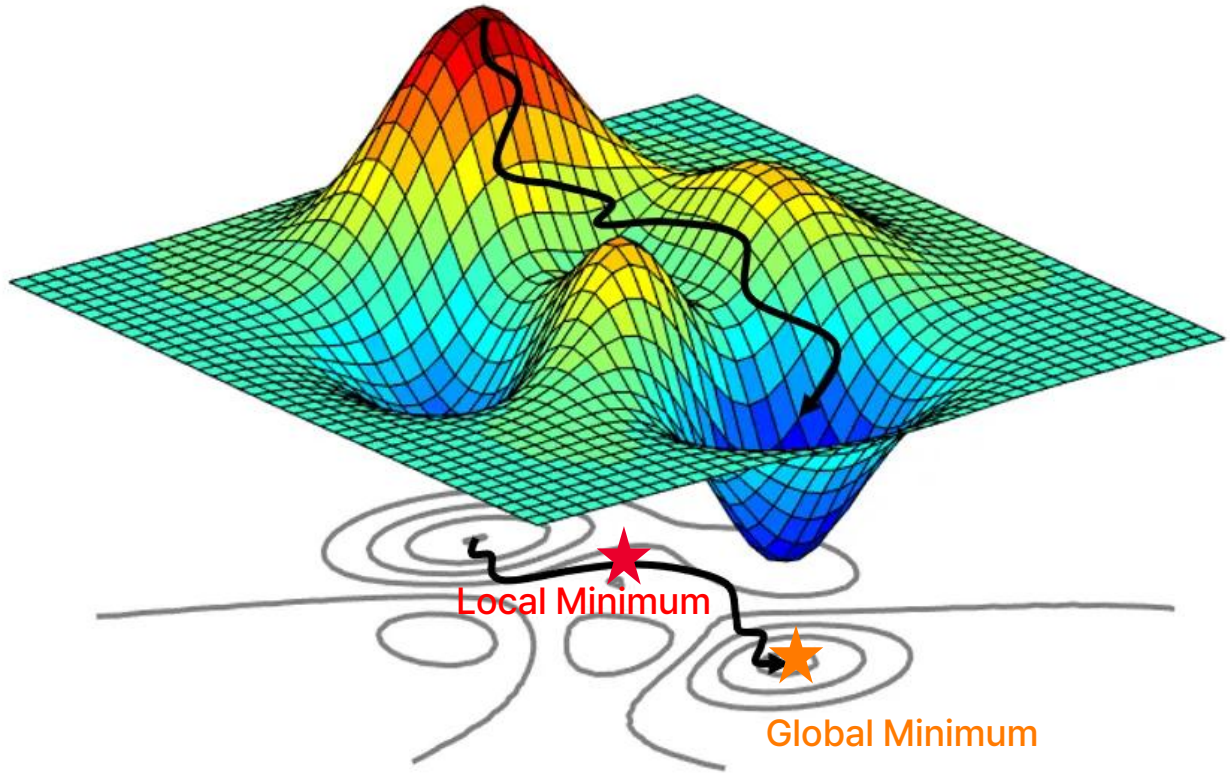
- W_t : 현재 위치 (현재 가중치)
- η : 한 걸음에 내려갈 거리 (학습률)
- $\frac{\partial E}{\partial W_t}$: 경사면의 경사. 현재 위치에서 얼마나 가파른지, 어느 방향인지 (기울기)
- $^{*}W_t$: 현재 위치에서 경사를 보고 한 걸음 내려간 다음 위치 (새로운 가중치)

[참고] Gradient Descent

가장 가파른 방향만 따라 내려가면 언젠가는 계곡에 도착하겠지만, 그 계곡이 진짜 우리가 도달해야 할 곳인가?

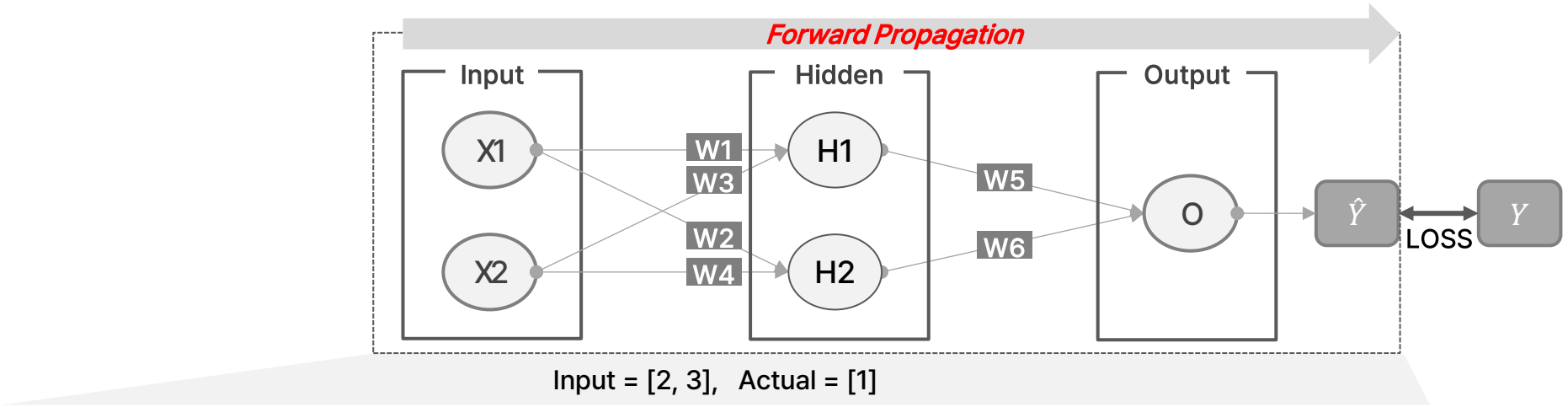
1. 목적 : 가장 낮은 곳(최소 손실값)을 찾는 것
2. 특징 : 지역 정보만 사용하여 산길 내려가기
3. 한계점
   - Local Minimum에 빠질 수 있음 (중간 계곡)
   - 경사면이 완만하면 느려짐
   - 출발 위치에 민감

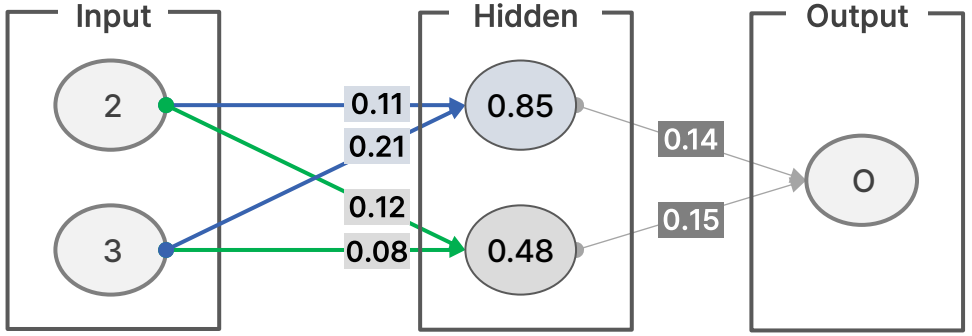
   → 항상 정답으로 가는 것은 아님



Source : https://fabiorosato.com/blog/navigating-success-as-gradient-descent

5. Forward vs Backward-propagation





FORWARD

$$[2 \quad 3] * \begin{bmatrix} 0.11 & 0.12 \\ 0.21 & 0.08 \end{bmatrix} = [0.85 \quad 0.48]$$

$$[0.85 \quad 0.48] * \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} = [\mathbf{0.191}]$$

(Details)

2 * 0.11 + 3 * 0.21 = 0.85
2 * 0.12 + 3 * 0.08 = 0.48
0.85 * 0.14 + 0.48 * 0.15 = 0.191

LOSS (Calculating Error)

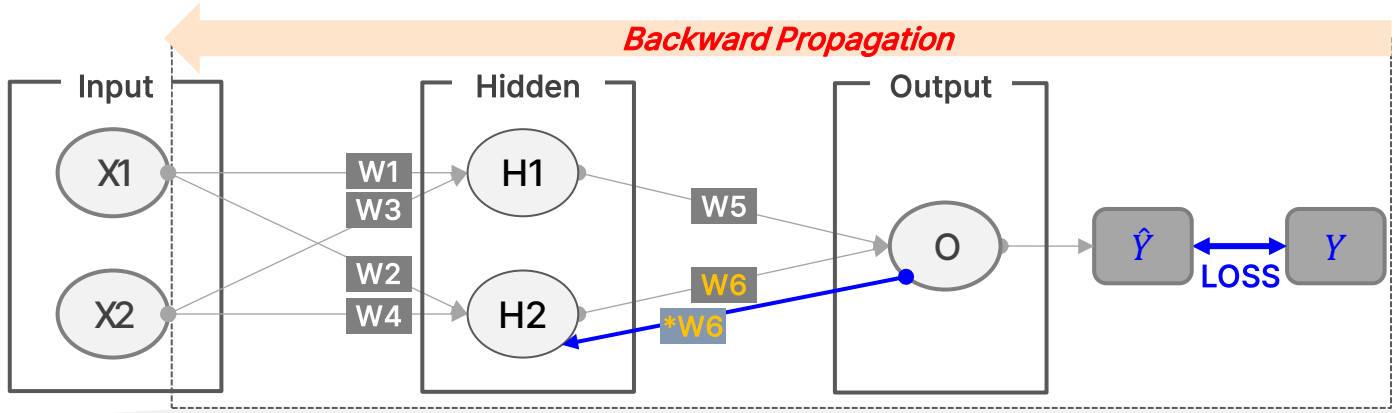
- 에러는 항상 양수
- 제곱함수의 미분에서 상수를 간단하게 만들기 위해 ½ 추가

$$Error = \frac{1}{2}(prediction - actual)^2$$

$$Error = \frac{1}{2}(0.191 - 1.0)^2 = 0.327$$

Source : https://hmkcode.com/ai/backpropagation-step-by-step/ (modified)

5. Forward vs Backward-propagation



역방향 논리 전개

prediction = O

prediction = $H_1W_5 + H_2W_6$, $\begin{cases} H_1 = X_1W_1 + X_2W_3 \\ H_2 = X_1W_2 + X_2W_4 \end{cases}$

prediction = $(X_1W_1 + X_2W_3)W_5 + (X_1W_2 + X_2W_4)W_6$

Loss 수식

$Error = \frac{1}{2}(prediction - actual)^2$

경사하강법의 Learning Rate

$^*W_t = W_t - \eta * \frac{\partial E}{\partial W_t}$

새로운 가중치 / 현재 가중치 / 학습률 / 기울기

$^*W_6 = W_6 - \eta * \frac{\partial E}{\partial W_6}$

$\frac{\partial E}{\partial W_6} = \frac{\partial Error}{\partial prediction} * \frac{\partial prediction}{\partial W_6}$

$= \frac{\partial}{\partial prediction}\left(\frac{1}{2}(prediction - actual)^2\right) * \frac{\partial}{\partial W_6}((X_1W_1 + X_2W_3)W_5 + (X_1W_2 + X_2W_4)W_6)$

$= (prediction - actual) * (X_1W_2 + X_2W_4)$

$= \Delta * H_2 = \Delta H_2$

- Hidden Layer, H_2의 출력 오차는 가중치(W_6) 업데이트가 요구됨
- 단계별 Backpropagation = 학습 과정에서 오차가 어떻게 전파/업데이트 되는지 확인
- **가중치 업데이트가 모델 성능 개선에 핵심!!**

5. Forward vs Backward-propagation

Backpropagation, 단계별 설명

1 가중치 업데이트 수식 (Gradient Descent)

$$^{*}W_6 = W_6 - \eta * \frac{\partial E}{\partial W_6}$$

- $\frac{\partial E}{\partial W_6}$ (기울기): 오차를 줄이기 위해 얼마나 가중치를 조정해야 하는지
- 가중치(W_6)에 대한 오차의 기울기 (미분)

2 Chain Rule 적용

$$\frac{\partial E}{\partial W_6} = \frac{\partial Error}{\partial prediction} * \frac{\partial prediction}{\partial W_6}$$

- 오차(E)는 예측값($prediction$)에 따라 달라지므로,
  오차를 예측값에 대해 미분하고, 예측값을 다시 가중치에 대해 미분함
- 오차 함수가 예측값과 가중치에 어떻게 의존하는지를 계산하는 과정

3 오차 함수 미분

$$\frac{\partial Error}{\partial prediction} = \frac{\partial}{\partial prediction}\left(\frac{1}{2}(prediction - actual)^2\right)$$

- 오차 함수는 예측값과 실제값의 차이의 제곱으로 정의
- 해당 식을 미분하면 예측오차 → 가중치를 얼마나 조정해야 하는지 결정

$$\frac{\partial}{\partial prediction}\left(\frac{1}{2}(prediction - actual)^2\right) = (prediction - actual)$$

4 예측값에 대한 미분

$$\frac{\partial prediction}{\partial W_6} = \frac{\partial}{\partial W_6}((X_1W_1 + X_2W_3)W_5 + (X_1W_2 + X_2W_4)W_6)$$

- 예측값($prediction$)은 가중치(W_6)에 따라 달라짐
- 해당 식을 미분하면 첫번째 항은 W_6와 무관하여 사라지고, 두번째 항만 남게 됨.

5 미분 결과 결합

$$\frac{\partial E}{\partial W_6} = (prediction - actual) * (X_1W_2 + X_2W_4)$$

- 3번과 4번 결과를 곱해서 최종적으로 가중치(W_6) 대한 오차 기울기 계산
- $(prediction - actual)$: 오차(delta)
- $(X_1W_2 + X_2W_4)$: 입력값과 이전 가중치 간의 조합.

6 수식 정리

$$\frac{\partial E}{\partial W_6} = \Delta * H_2 = \Delta H_2$$

- $(X_1W_2 + X_2W_4) = H_2$ 로 치환하여 수식 정리
- H_2 : 히든 레이어의 출력
- 최종적으로 가중치(W_6)가 어떻게 조정되는지를 확인

[참고] Backpropagation 쉽게 이해하기

출발해서 내려가는 것만 중요한 것이 아니고, 내려가다 잘못됐으면 올라가서 수정해야 진짜 좋을 길을 찾을 수 있다!!

- 산 아래(Global Minimum)로 내려오고 싶었는데, 길이 잘못됐다는 걸 알게 된 경우
- 잘못된 경로를 되짚어 올라가면서 (Backprop)
- 어디서부터 잘못된 건지 찾고 (기울기 계산, 업데이트)
- 조금씩 수정해서 다시 더 나은 길로 가는 과정

Backpropagation

① 내려가면서 길 찾기 (Gradient Descent)

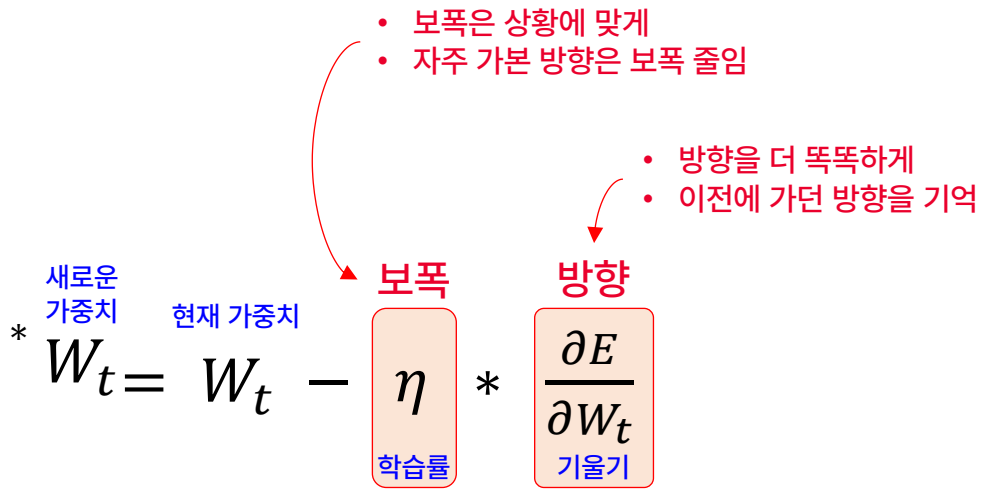
② 잘못된 길 찾으면 되짚어 올라가기 (오차 역전파)

③ 올라가면서 틀림 지점(가중치) 수정하기

단계	설명	비유
오차 계산	• 실제값 – 예측값 결과 비교해서 오차 계산	• 산 아래 도착했는데 목표랑 다름
오차 전파	• 오차가 어디서부터 발생했는지 거꾸로 따라 올라감	• 길을 되짚어 올라가기 시작!
기울기 계산	• 각 지점(가중치)에서 방향과 조절 정도 확인	• 돌이 많아서 느려진 건지, 길이 꼬인 건지 파악
가중치 업데이트	• 계산한 기울기(방향) 따라 조금씩 수정	• 다시 내려갈 때 더 좋은 길로!

[참고] 가중치 업데이트, 정리하기

경사하강법의
Learning Rate



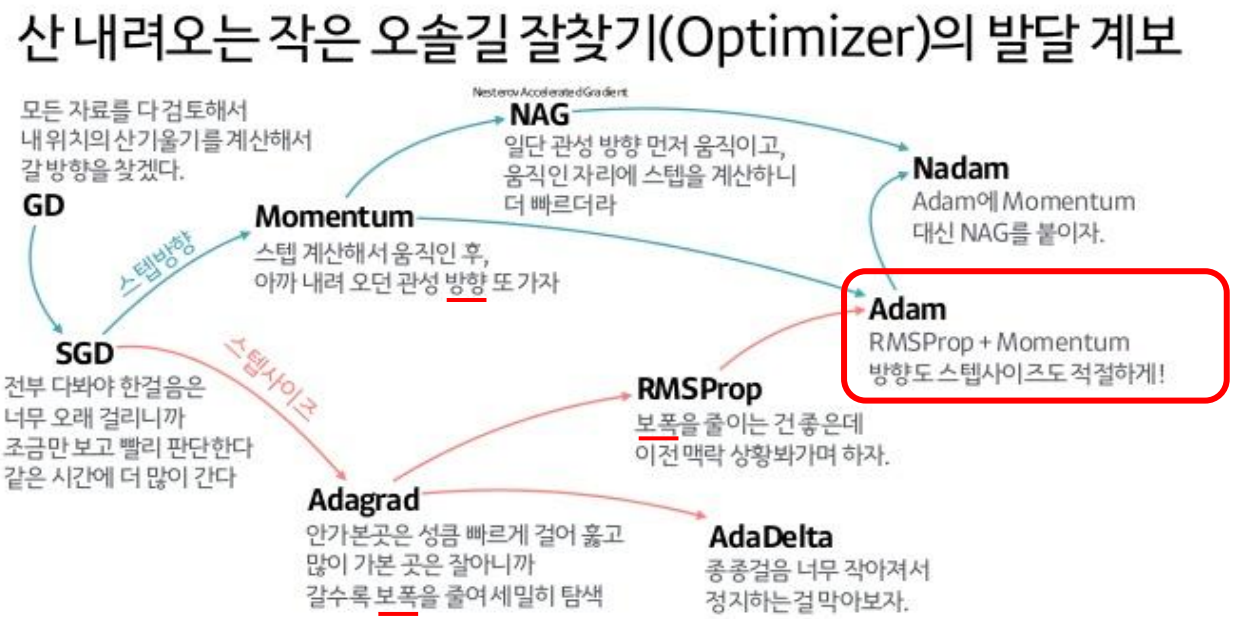
$$^{*}W_t = W_t - \eta * \frac{\partial E}{\partial W_t}$$

- W_t : 현재 위치 (현재 가중치)
- η : 한 걸음에 내려갈 거리 (학습률)
- $\frac{\partial E}{\partial W_t}$: 경사면의 경사. 현재 위치에서 얼마나 가파른지, 어느 방향인지 (기울기)
- $^{*}W_t$: 현재 위치에서 경사를 보고 한 걸음 내려간 다음 위치 (새로운 가중치)

6. Optimizer

Key Point

Adam은 SGD와 비슷하게 기울기를 업데이트 하지만, 각 파라미터 마다 다른 학습률을 자동으로 저장하는 알고리즘



- Adam = 방향 기억(Momentum) + 상황별 보폭 조절(Adagrad/RMSProp) ➔ ***두 가지 방안을 결합한 "Adam"***
- 더 빠르고 흔들림 없이 Global minimum 으로 수렴

[참고] Adam Optimizer 쉽게 이해하기

산을 내려갈 때, 이전 경험을 기억하면서 더 똑똑하게 걷는 방법

- Gradient Descent는 단순하게 현재 경사(기울기)만 보고 내려감
- Adam은 내려가면서 생긴 피로도와 이전 경로의 방향성까지 고려해서 더 빠르게, 덜 흔들리며 내려감
  → 지형이 울퉁불퉁해도 효과적!

비유적 표현

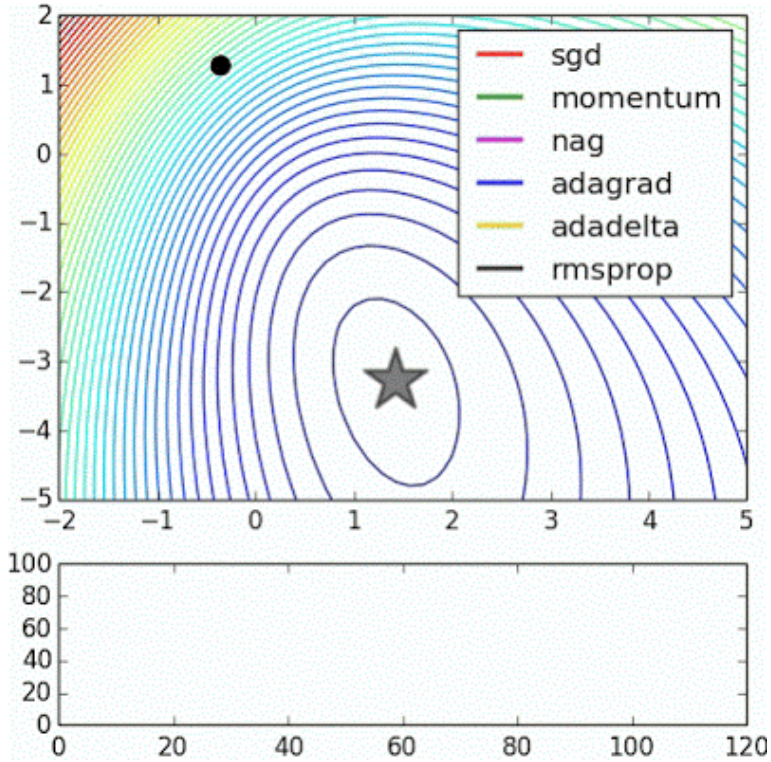
- Gradient Descent는 등산객이 현재 위치에서 경사만 보고 움직이는 방식
- Adam은 등산객이 다음을 고려함:
  - 지금까지 어느 쪽으로 가고 있었나 : 방향 기억 – 기울기 평균(이전 방향들의 누적)
  - 이 방향이 얼마나 들쭉날쭉/가파른가 : 변동 크기 – 기울기 제곱의 평균

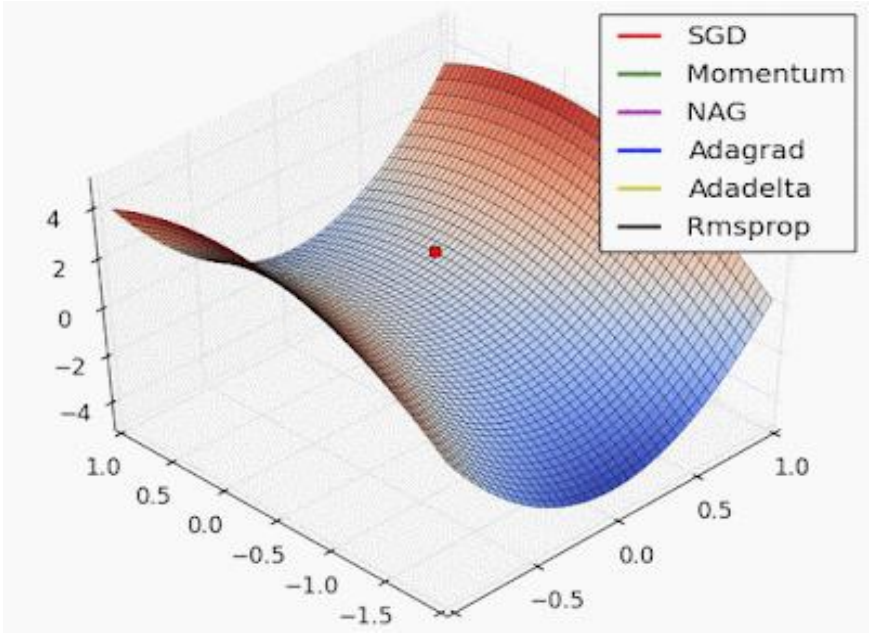
Adam Optimizer 흐름

① 현재 기울기 계산 : 지금 발 밑이 얼마나 가파른지 확인
② 1차 계산 (방향) : 지금까지 내려온 길을 종합해 속도/방향 정하기
③ 2차 계산 (변동) : 지금까지 내딛은 발걸음들이 발걸음들이 얼마나 들쭉날쭉했는지 누적으로 기록
④ 업데이트 : 넘어지지 않도록 균형 잡으면서 빠르게 내려가기

6. Optimizer

Alec Radford's animations for optimization algorithms (2015)





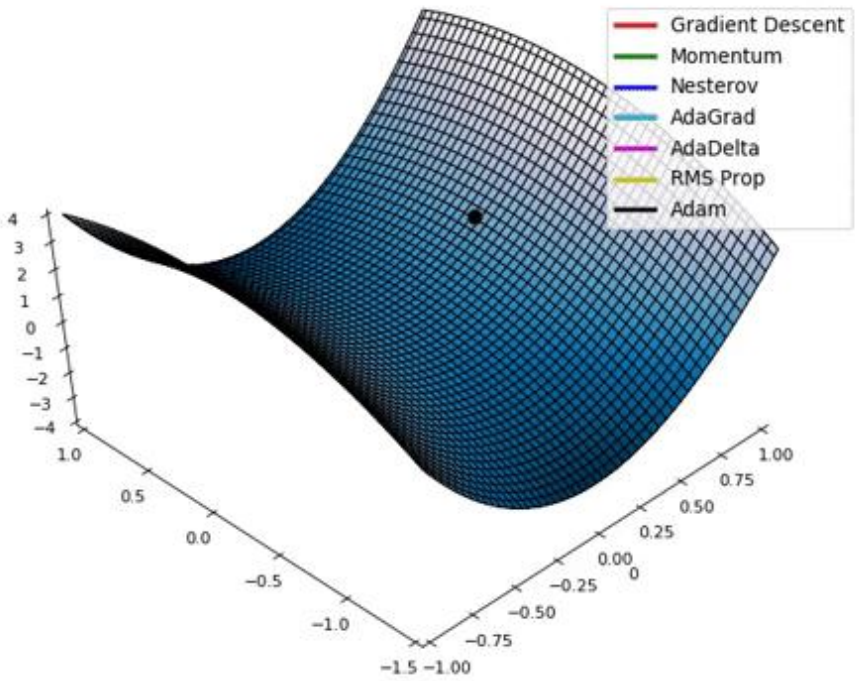
[Left] The error surface. The trajectories show the dynamics of minibatches on noisy data. The bottom chart is an accuracy plot.

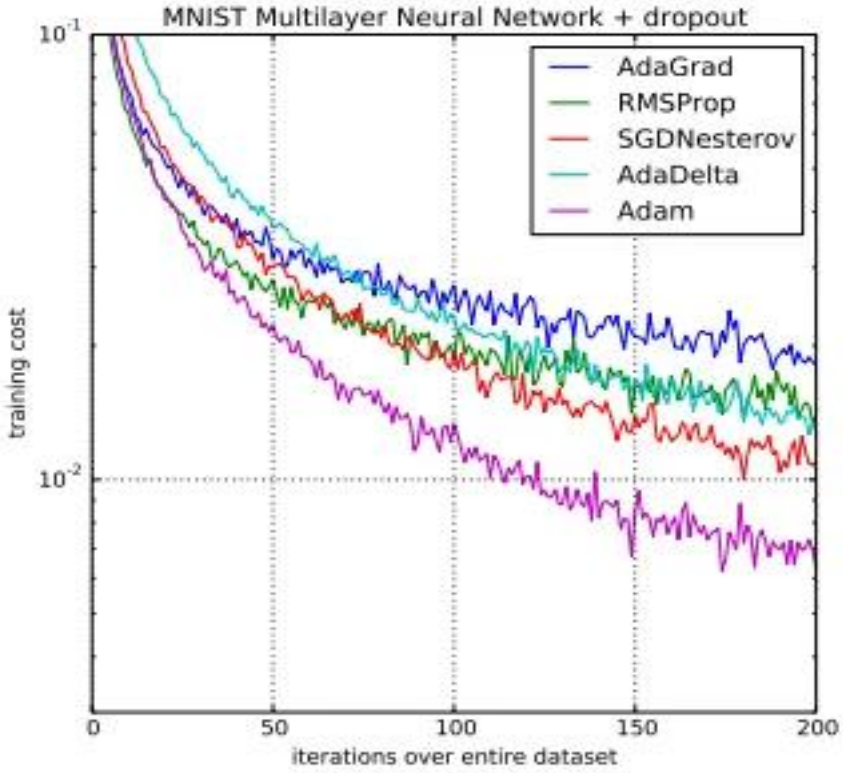
[Right] Behavior around a saddle point.

Source : http://www.denizyuret.com/2015/03/alec-radfords-animations-for.html

6. Optimizer

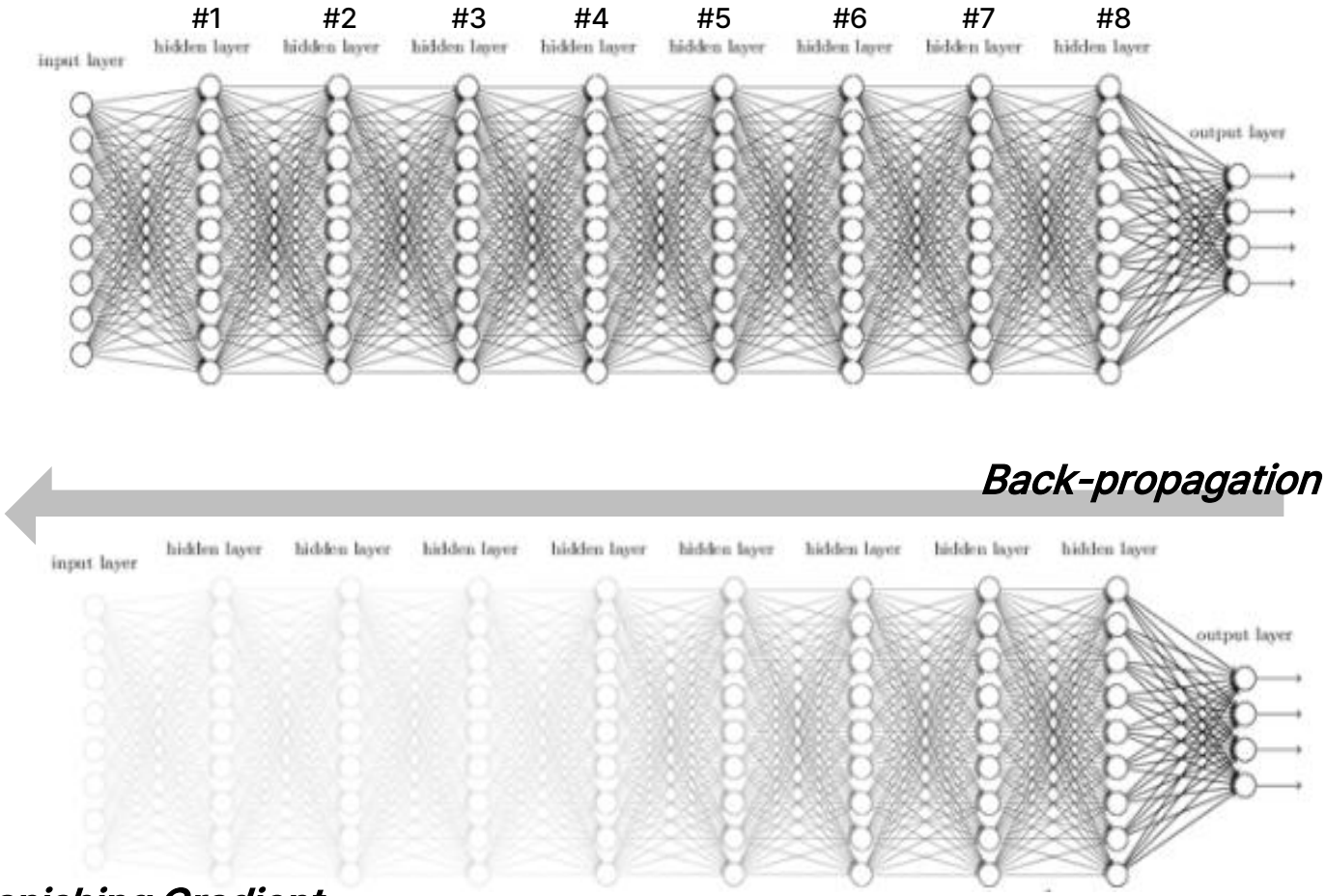
ADAM : A Method For Stochastic Optimization (2017)





Source : https://community.deeplearning.ai/t/difference-between-rmsprop-and-adam/310187/2

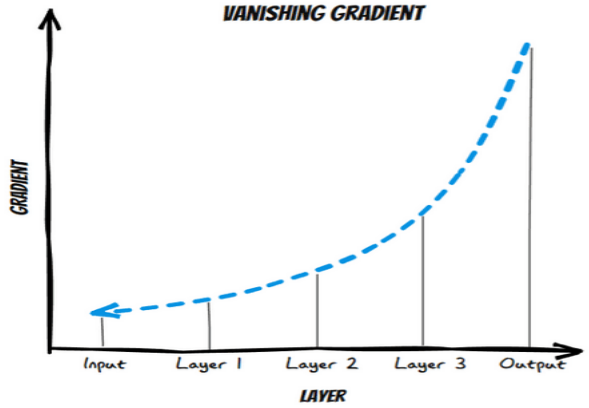
7. Gradient Vanishing

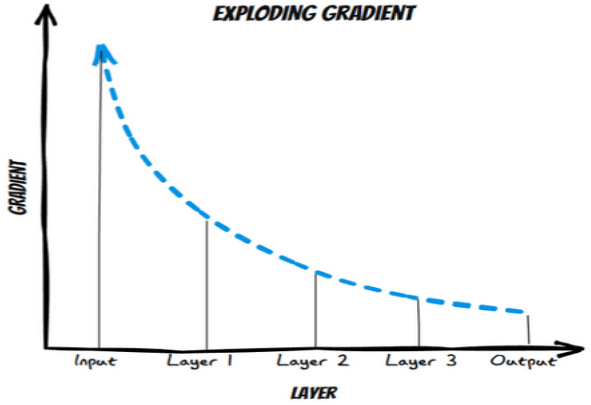


Source : https://www.slideshare.net/Takayosi/miru2014-tutorial-deeplearning-37219713

7. Gradient Vanishing & Exploding

Backpropagation 과정에서 기울기 크기가 크기가 너무 작거나 혹은 커지면서 학습이 제대로 이루어지지 않는 현상





현상	• Gradient가 레이어를 거칠수록 점점 작아지는 현상 • 결과적으로 신경망의 초기 레이어들이 거의 학습되지 않음 → **전체 모델 성능 하락**	• Gradient가 레이어를 거칠수록 지나치게 커지는 현상 • 가중치 업데이트가 너무 크게 일어나면서 학습이 제대로 되지 않음 → **모델이 최적의 값에 수렴하지 못하고, 발산할 수 있음**
발생 원인	• **Sigmoid, Tanh 함수에서 주로 발생** • 0에 가까운 경우가 많아, 역전파 과정에서 기울기가 계속 곱해지면 점점 더 값이 작아지게 됨 • 작은 기울기들이 계속 작아져서 마지막에는 거의 0에 수렴 → 가중치가 업데이트되지 않고 학습이 멈추게 됨	• **주로 신경망이 매우 깊거나 초기화가 잘못된 경우 발생** • 특정 레이어에서 기울기가 매우 큰 값을 가지게 되면, 역전파 과정에서 더 커져 폭발적인 기울기 값이 발생하게 됨
해결 방법	• ReLU : 양수 영역에서 기울기가 일정하여 Vanishing 문제 해결 • Batch Normalization	• Gradient Clipping : 기울기가 일정값 이상 커지지 않도록 제한하여 Exploding 방지 (임계값을 넘으면 해당 값으로 제한)

[참고] Gradient Descent

	Gradient Descent	Learning Rate	Gradient Vanishing	Gradient Clipping	Gradient Decay
대상	모델의 가중치 업데이트 방법	가중치 업데이트 속도	기울기(Gradient)	기울기(Gradient)	가중치(Weight)
목적	손실함수의 기울기(Gradient) 따라 가중치 업데이트	학습속도 조절, 기울기 크기에 따라 가중치 변화량 조절	신경망이 깊을 때 기울기가 사라져 학습이 멈추는 현상	기울기 폭발 방지, 학습 안전성 유지	과적합 방지, 모델의 일반화 성능 향상
작동 방식	손실함수의 기울기를 계산하여 가중치를 반대 방향으로 업데이트	학습률(Hyper-parameter) 따라 가중치 변화량 결정	역전파 과정에서 기울기가 점차 작아져 레이어에 도달하기 전에 사라짐	기울기가 설정된 임계값을 넘으면 그 값을 제한	손실함수에 가중치 크기에 대한 패널티 추가
장점	단순한 원리로 가중치 최적화	학습 속도를 조절하여 수렴을 가능하게 함 (Global Minimum)	학습이 진행되지 않는 현상 설명	학습의 안정성 보장, 기울기 폭발 방지	과적합 방지로 일반화 성능 향상
단점	Learning Rate 설정 따라 너무 느리게 수렴하거나 발산할 수 있음	잘못 설정되면 학습 실패 원인이 될 수 있음	기울기가 0에 가까워져 학습이 거의 이루어지지 않음	너무 작은 임계값 설정시 학습속도가 느려짐	정규화 정도가 너무 크면 모델 성능 저하

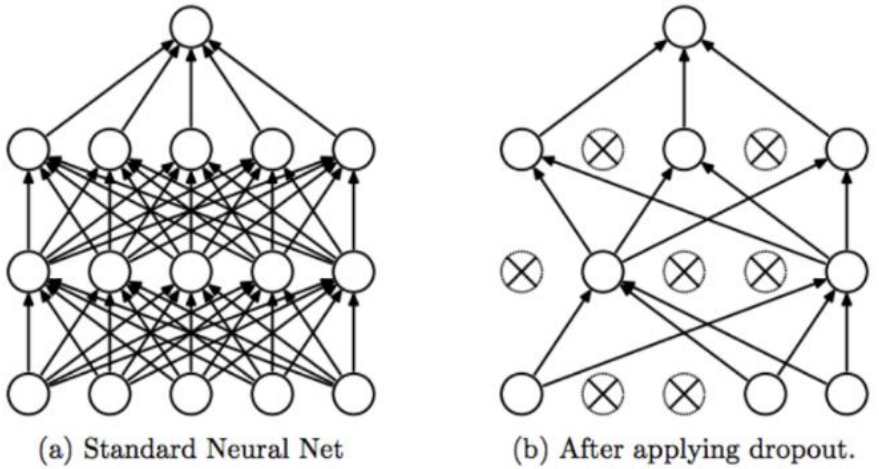
비유를 통한 설명 :

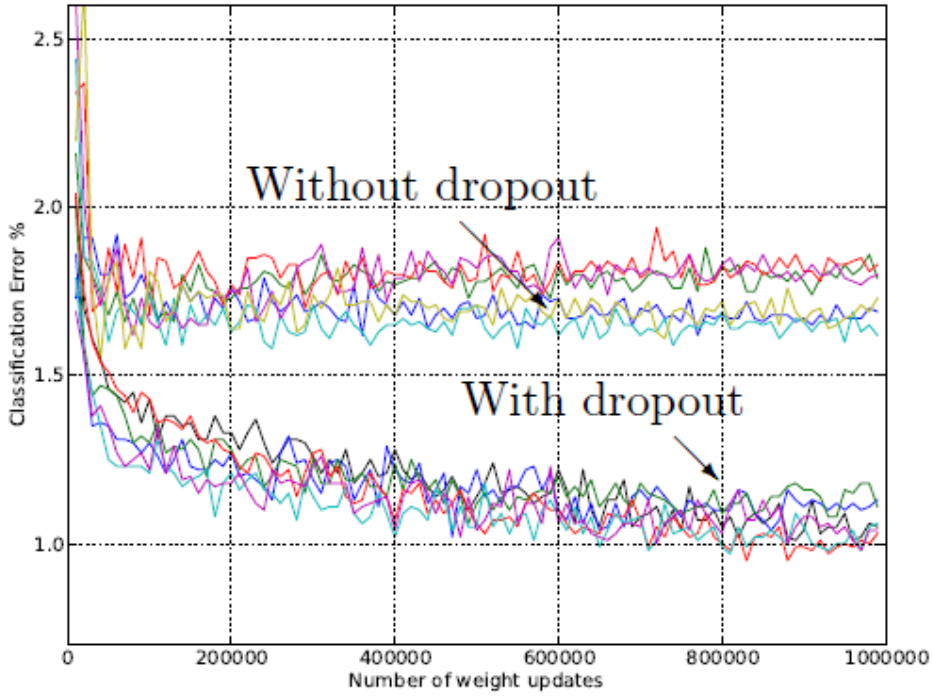
- Gradient Descent : 산을 내려가면서 최적의 지점을 찾는 과정 (손실 함수라는 산을 내려가면서 최소점을 찾기 위해 경사를 따라 이동함)
- Learning Rate : 산을 내려나는 등산 중에 걸음의 크기를 적절히 조절해 안정적으로 내려가는 것
  (너무 큰 걸음을 내딛으면 넘어지거나(발산) 목표를 지나칠 수 있고, 너무 작은 걸음을 내딛으면 시간이 오래 걸림)
- Gradient Vanishing : 짙은 안개로 길이 보이지 않는 상황처럼, 기울기가 사라져 학습이 잘 안되는 것
- Gradient Clipping : 가파른 경사를 줄로 제한해 넘어지지 않도록 하는 것
- Gradient Decay : 배낭의 무게를 줄여 가볍게 등산하는 것

8. Dropout

Key Point

Hidden Layer의 뉴런을 무작위로 골라 삭제하면서 학습 → 골고루 학습 효과





[주의]

- Training 단계 → 삭제할 뉴런을 무작위로 선택
- Testing 단계 → 학습한 모델의 결과를 확인해야 함으로
  모든 뉴런에 신호를 전달하여 확인

AI Campus 데이터분석 및 AIOps > Part B – Deep Learning

3. 데이터 구조

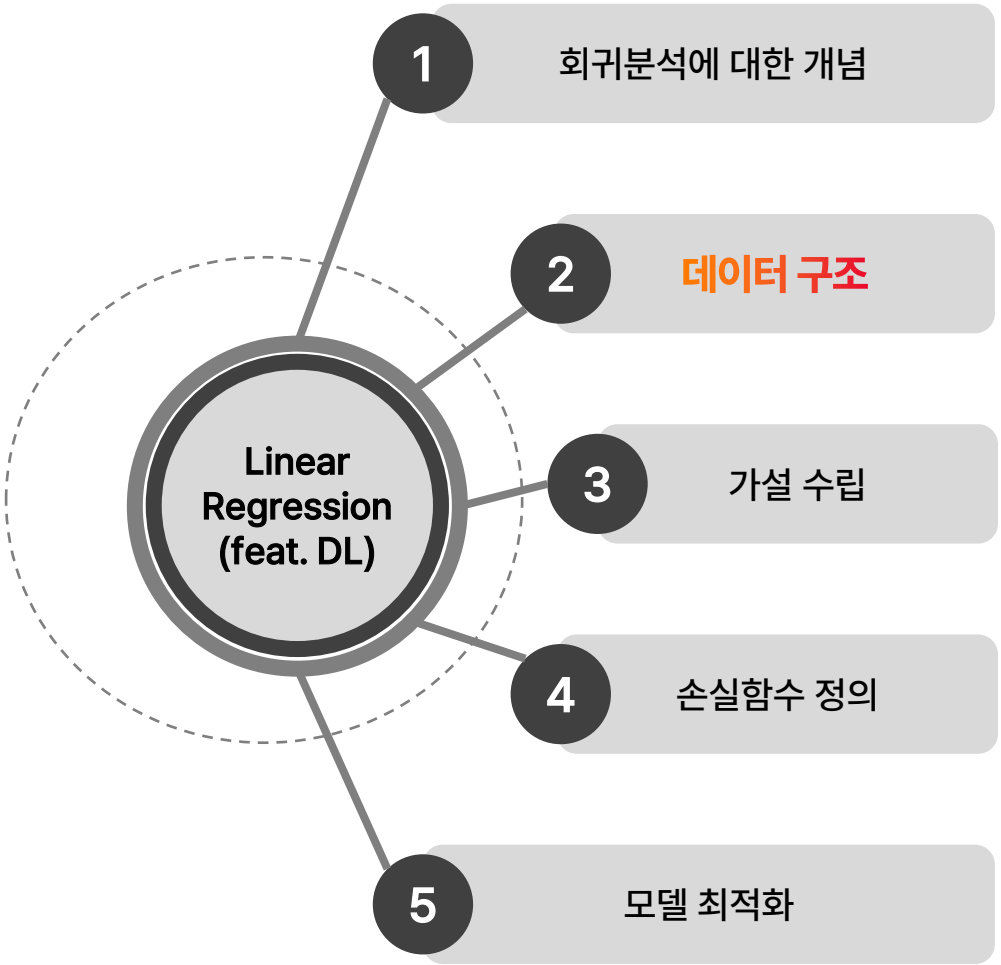
- 딥러닝에 적합한 데이터 형태
- Tensor Shape 이해

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

Linear Regression 재해석

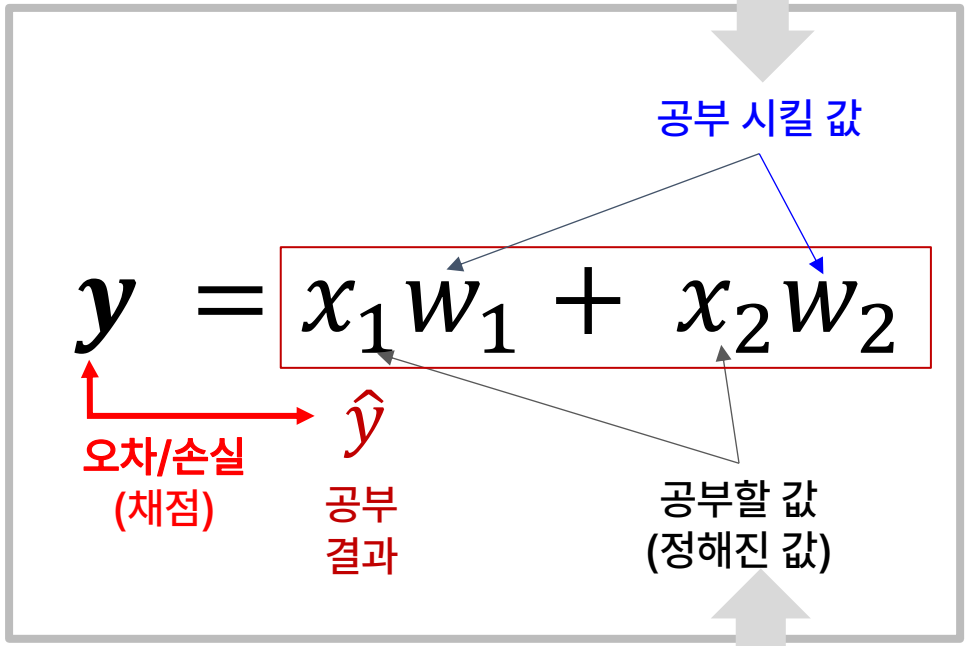
Key Point



- **회귀분석에 대한 개념 이해**
  - 회귀분석의 정의 및 활용 목적
  - 딥러닝 적용 위해 필요한 추가 절차
- **Data Definition, 데이터 구조에 대한 이해**
  - 딥러닝에 적합한 데이터 형태
  - Tensor Shape 이해
- **수립한 가설 기반으로 모델 개발**
  - 가설 수립 방법의 이해
  - 선형모델 또는 딥러닝 신경망 네트워크 아키텍처 구조 적용
- **손실함수 정의 및 계산**
  - 모델의 적합 여부를 평가하기 위한 기준의 손실 함수 정의/적용
  - 목적별 손실함수 이해
- **최적화 개념 및 방법**
  - Gradient Descent 개념 이해
  - 최적화 종류 및 종류별 장단점 이해/적용

Deep Learning Goal

Key Point



무엇을, 언제까지 공부 시키지?

- 정답(Y)와 최대한 가까워지도록 **가중치**를 학습
  → **Gradient Descent** (Loss Function)
- **Overfitting** 발생하기 직전까지 학습
  → **Dropout** (학습 중단, early stopping)

학습 데이터가 큰데 어떻게 하지?

- **작은 셋으로 나누어** 학습 시키자
  → **Batch, Mini-batch** (데이터 잉여성 기반)
- **여러 번** 하면 데이터 특징을 더욱?!
  → **Epoch** (학습 데이터 소진 단위)

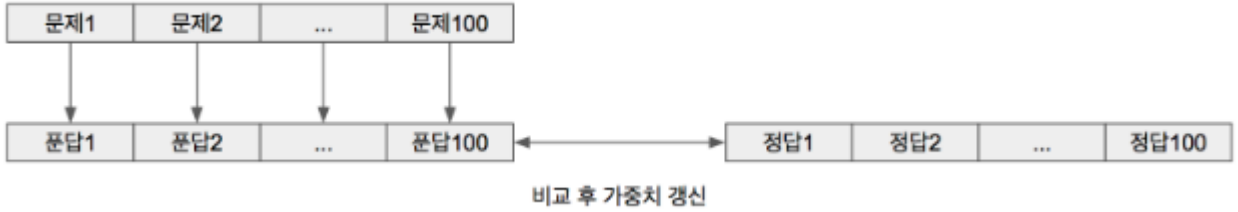
※ **잉여성**: input이 바뀌어도 output은 바뀌지 않는다

1. Batch

학습 데이터셋은 어떻게 구성하지?

- **Batch Size = 100**

  → 100문제를 한번에 다 풀고 정답 비교 → 100개 돌리고 가중치 1번 업데이트



Batch Size가 작을수록
자주 가중치 업데이트!

- **Batch Size = 10**

  → 10문제씩 풀고 정답 비교 → 10개씩 돌리고 가중치 업데이트 → 10번 가중치 업데이트

학습을 더 꼼꼼하게~



1. Batch & Epoch

데이터 특징이 잘 반영될 수 있도록 어떻게 학습 시키지?

- **Batch Size = 10**

  → 10문제씩 풀고 정답 비교 → 10개씩 돌리고 가중치 업데이트 → 10번 가중치 업데이트



- **Epoch : 시험문제 1set을 몇 번 반복해서 풀까**

  Epoch = 20

  → 시험문제 1set * 20번

  → 과거 풀었던 것 보다 학습 결과 향상을 기대할 수 있음

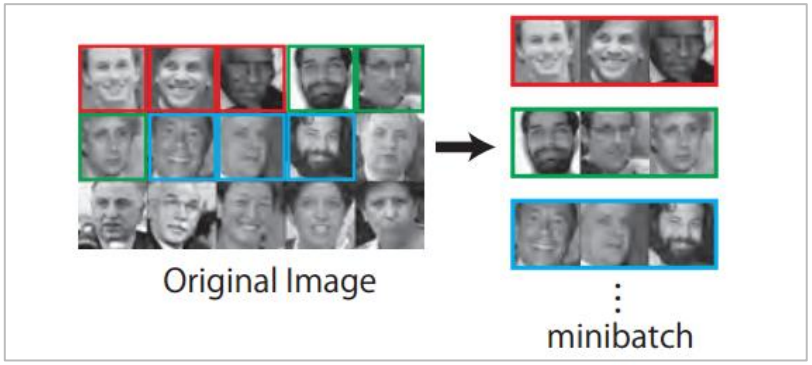
반복 학습 효과.

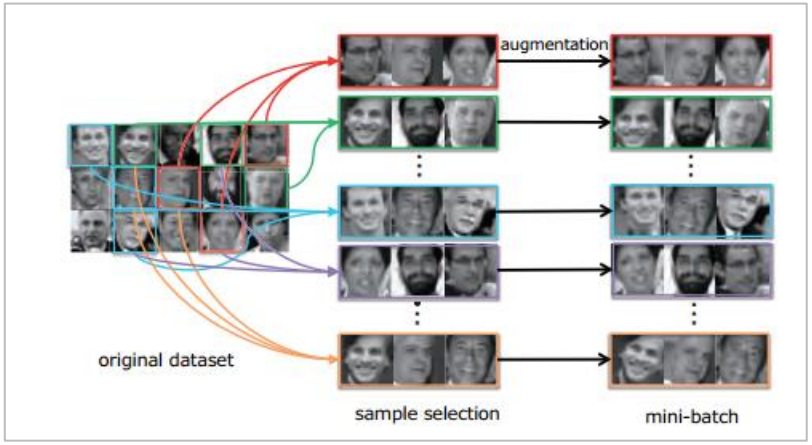
그러나 과도한 반복은 Overfitting !!

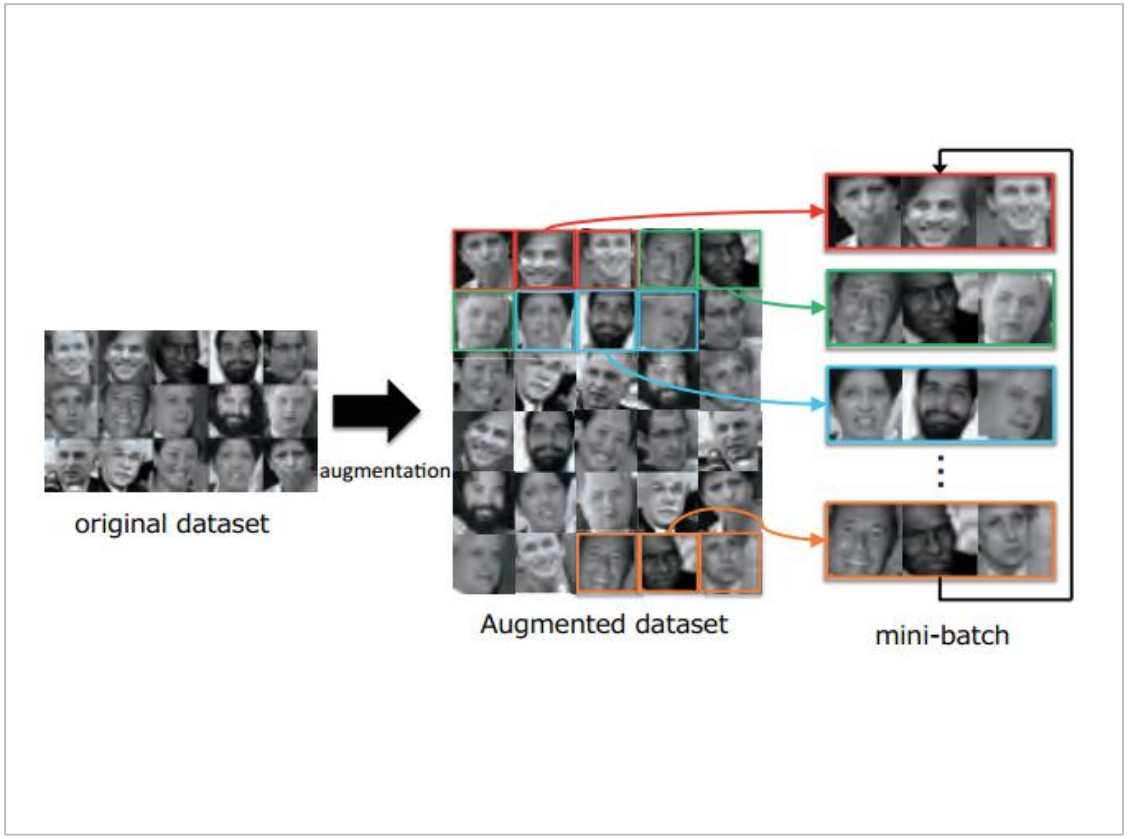
[참고] Mini Batch

전체 데이터에서 작은/소량 데이터로 나누고, 목적에 맞게 Data Augmentation 적용

Deep Convolutional Neural Network による顔器官点検出における最適なミニバッチ作成方法 (2015)







2. Epoch, Batch, Mini-Batch

```
import torch
from torch.utils.data import DataLoader, TensorDataset

# (1) 더미 데이터셋 생성 (총 20개의 샘플, 각 샘플은 2개의 피처)
X = torch.arange(1, 41).reshape(-1, 2).float()  # shape: (20, 2)
y = torch.arange(1, 21).float().reshape(-1, 1)   # shape: (20, 1)

# (2) PyTorch Dataset 구성
dataset = TensorDataset(X, y)

# (3) DataLoader로 Mini-Batch 구성 (batch_size=5이면 mini-batch 4개)
batch_size = 5
loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

# (4) Epoch 설정
num_epochs = 3

# (5) 학습 루프 시작
for epoch in range(num_epochs):
    print(f"\n Epoch {epoch+1}/{num_epochs}")

    for batch_idx, (features, targets) in enumerate(loader):
        print(f"\n Mini-Batch {batch_idx+1}")
        print(f" - Features:\n{features}")
        print(f" - Targets:\n{targets}")
```

- **Epoch**
  - 전체 데이터셋을 한 번 학습
    → 파라미터를 한 번씩 업데이트하는 과정
  - (e.g.) 20개 데이터 : 1 epoch에서 20개 모두 사용
    num_epcohs = 3 : 위 과정을 3번 반복

- **Batch**
  - 학습 시 한번에 처리하는 데이터 묶음
  - (e.g.) 총 20개 데이터셋을 한번에 모두 사용
    : batch_size = 20 → 1 epoch 당 배치 1개

- **Mini-Batch**
  - Batch 중에서, batch_size가 전체 데이터보다 작은 경우
  - (e.g.) 총 20개 데이터셋 & batch_size = 5 :
    1 epoch 당, 4개의 mini-batch로 학습

➔ 메모리 절약, 학습 속도 향상, gradient 안정성 확보

2. Epoch, Batch, Mini-Batch

```
import torch
from torch.utils.data import DataLoader, TensorDataset
```

```
# (1) 더미 데이터셋 생성 (총 20개의 샘플, 각 샘플은 2개의 피처)
X = torch.arange(1, 41).reshape(-1, 2).float()  # shape: (20, 2)
y = torch.arange(1, 21).float().reshape(-1, 1)   # shape: (20, 1)
```

```
원시 데이터
X: (20, 2) ─┐
y: (20, 1) ─┘          →  TensorDataset(X, y)
                            = [(x0,y0), (x1,y1), …, (x19,y19)]  총 20쌍
```

```
# (2) PyTorch Dataset 구성
dataset = TensorDataset(X, y)
```

```
# (3) DataLoader로 Mini-Batch 구성 (batch_size=5이면 mini-batch 4개)
batch_size = 5
loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
```

```
batch_size = 5, shuffle=True
┌──────────────── 전체 20샘플 ────────────────┐
| Batch #0 | Batch #1 | Batch #2 | Batch #3 |
| 5쌍      | 5쌍      | 5쌍      | 5쌍      |   → 총 4개 mini-batch
└──────────────────────────────────────────┘
(예: 인덱스 범위 예시 — 0~4, 5~9, 10~14, 15~19  *shuffle=True면 순서는 섞임)
```

```
# (4) Epoch 설정
num_epochs = 3

# (5) 학습 루프 시작
for epoch in range(num_epochs):
    print(f"\n Epoch {epoch+1}/{num_epochs}")

    for batch_idx, (features, targets) in enumerate(loader):
        print(f"\n Mini-Batch {batch_idx+1}")
        print(f" - Features:\n{features}")
        print(f" - Targets:\n{targets}")
```

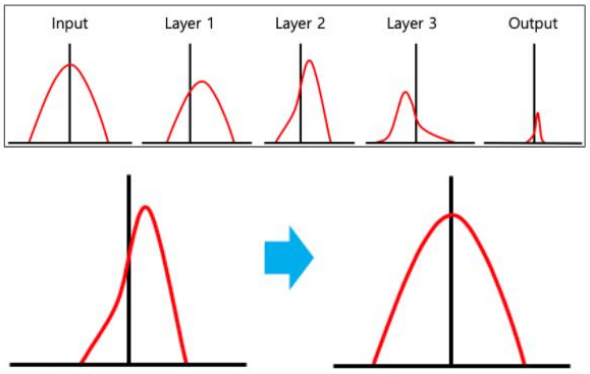
```
num_epochs = 3

Epoch 1
  ├─ Mini-Batch 0  (파라미터 업데이트 #1)
  ├─ Mini-Batch 1  (업데이트 #2)
  ├─ Mini-Batch 2  (업데이트 #3)
  └─ Mini-Batch 3  (업데이트 #4)

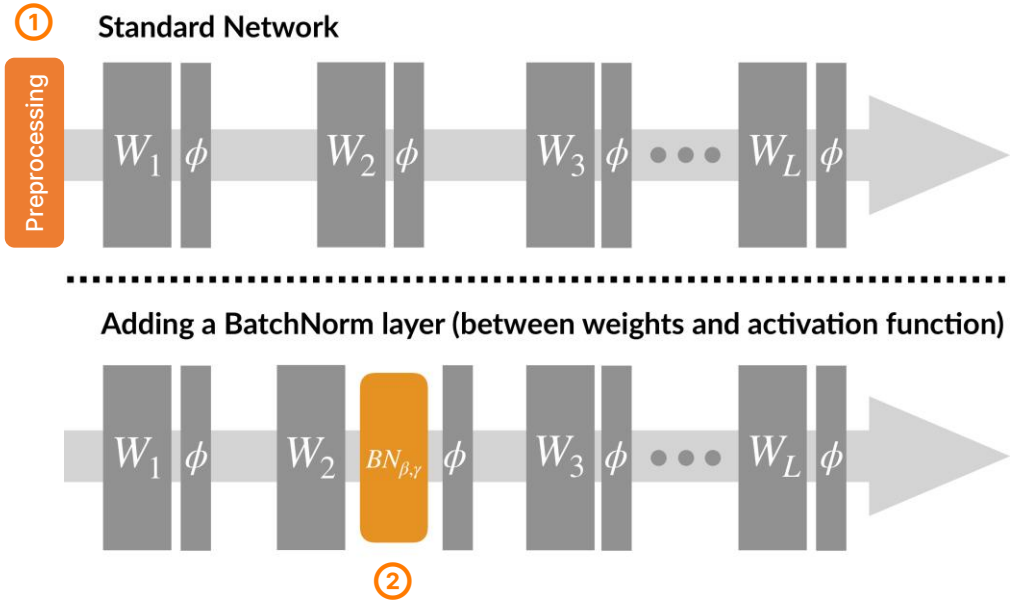
Epoch 2
  ├─ MB 0  (업데이트 #5)
  ├─ MB 1  (업데이트 #6)
  ├─ MB 2  (업데이트 #7)
  └─ MB 3  (업데이트 #8)

Epoch 3
  ├─ MB 0  (업데이트 #9)
  ├─ MB 1  (업데이트 #10)
  ├─ MB 2  (업데이트 #11)
  └─ MB 3  (업데이트 #12)
```

3. Batch Normalization



- BN은 각 층의 입력을 정규화해 학습을 빠르고 안정적으로 만듦
- 손실 지형을 매끄럽게 해 기울기를 안정시키는 것이 핵심
- 그 결과 기울기 안정화(Vanishing 완화) · 더 큰 학습률 사용 가능 · 빠르고 안정적 수렴



- 기대효과:
  - Gradient 흐름 개선 : 입력 분포가 안정되면서 Vanishing 완화
  - 학습 속도 향상 : 더 큰 학습률(learning rate) 사용 가능
  - 학습 안정성 확보 : 각 층의 입력 분포가 일정하게 유지
  - Regularization 효과 : 과적합 방지 효과



[Optimization Landscape, 학습 공간]

BN은 공간을 매끄럽게 처리해 주어 경사가 지나치게 요동치거나 평평해지는 문제를 줄일 수 있음. 덕분에 gradient가 안정적으로 수렴하고, 더 큰 학습률에서도 학습이 수월하게 진행될 수 있음 → 모델은 빠르고 안정적으로 수렴할 수 있음

Source : https://gradientscience.org/batchnorm/ (modified), https://velog.io/@choiking10/Batch-Normalization에-대해서-알아보자

[참고] Data Structure

Scalar

- 구성인자가 1개인 벡터

```
a    1
b    "bbb"
```

Vector

- 1차원으로 배열된 데이터 구조
  (모두 동일한 유형의 자료)

```
x    num [1:3] 1 2 3
y    chr [1:3] "A" "B" "C"
```

Factor

- 범주형 자료(명목형, 순서형)를 위한 구조

```
Ord.factor w/ 3 levels "Low"<"Medium"<..: 3 1
chr [1:5] "High" "Low" "Low" "Medium" "High"
```

Array

- 2차원 이상의 데이터 구조

```
, , 1

     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6

, , 2

     [,1] [,2] [,3]
[1,]    7    9   11
[2,]    8   10   12
```

List

- 여러 유형, 여러 형태의 데이터를 묶은 형태

```
[[1]]
[1] "High"   "Low"    "Low"    "Medium" "High"

[[2]]
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    4    7   10   13
[2,]    2    5    8   11   14
[3,]    3    6    9   12   15
```

Matrix

- 2차원으로 배열된 데이터 구조
  (모두 동일한 유형의 자료)

	V1	V2	V3	V4	V5
1	1	4	7	10	13
2	2	5	8	11	14
3	3	6	9	12	15

Data Frame

- 2차원으로 배열된 데이터 구조
  (※ 각 열마다 다른 형태 데이터 지원)

	cust_id	gender	age	ord_amt
1	c01	M	30	50000
2	c02	F	28	0
3	c03	NA	24	120000

4. Tensor

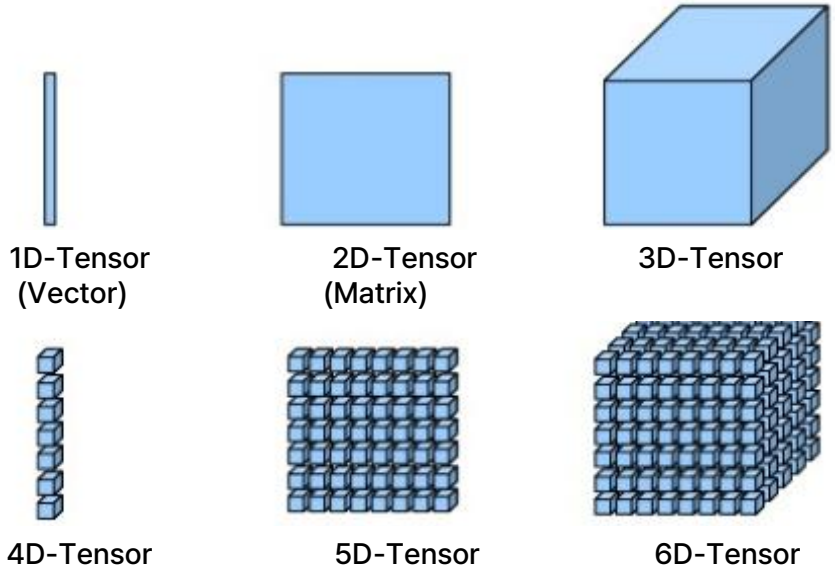
용어의 이해

벡터. Vector : 1차원 배열

텐서. Tensor

- **벡터의 개념을 확장**한 기하학적인 양
- 두 개 이상의 벡터 조합으로 구성된 물리량

- 딥러닝은 선형 관계를 넘어 복잡한 **비선형 관계를 학습**해야 하는데, 이를 위해 **Tensor는 데이터를 다차원 공간으로 표현** 하여 더 많은 정보를 효율적으로 학습할 수 있도록 함



5. 차원별 텐서

0D-Tensor, Scalar

- 구성인자가 1개인 Vector
- 예시 : x = 1

1D-Tensor, Vector

- 1차원 배열로 단순히 숫자들이 일렬로 나열된 형태
- 예시 : x = [1, 2, 3]

2D-Tensor, Matrix

- 2차원 배열로 행과 열로 구성된 형태
- 각 행이 데이터 포인트를 나타내고, 각 열이 해당 데이터 포인트의 특정을 나타낼 수 있음
- 예시 : $x = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

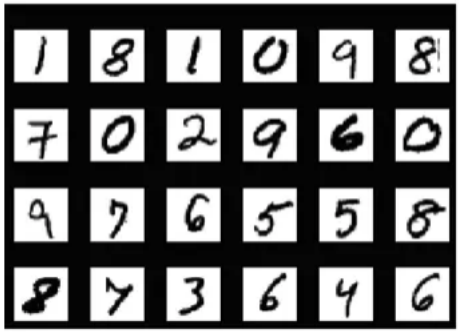
3D-Tensor

- 3차원 배열로 여러 행렬을 하나의 묶음으로 포함
- 대표적으로 컬러 이미지가 있는데 [가로, 세로, 채널]의 형태로 표현되고, 색상은 RGB 채널 정보를 담고 있음
- 예시 : $x = \begin{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \end{bmatrix}$

4D-Tensor

- 4차원 배열로 여러 3D 텐서를 포함하는 형태
- 대표적으로 동영상 데이터로 [프레임수, 가로, 세로, 채널] 형태로, 3D 텐서들이 시각 축을 따라 나열된 형태임
- 예시 : $x = \begin{bmatrix} \begin{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \end{bmatrix}, \begin{bmatrix} \begin{bmatrix} 9 & 8 \\ 7 & 6 \end{bmatrix}, \begin{bmatrix} 5 & 4 \\ 3 & 2 \end{bmatrix} \end{bmatrix} \end{bmatrix}$

5. 차원별 텐서



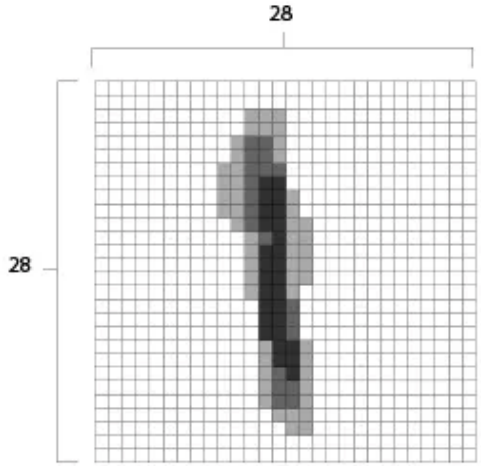


fig-4 single-data pixel

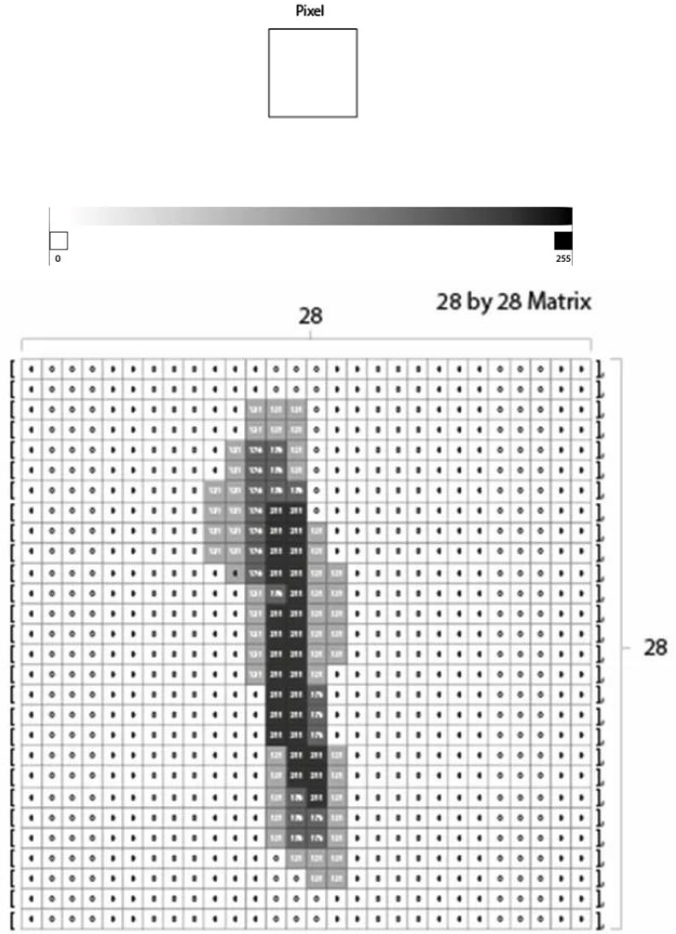
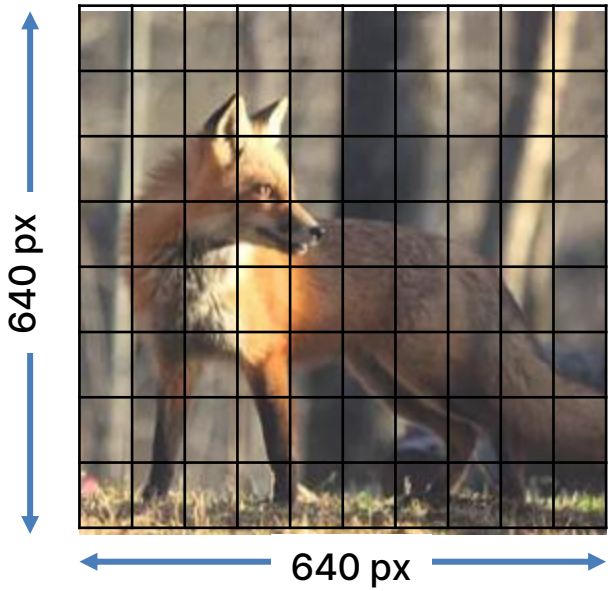
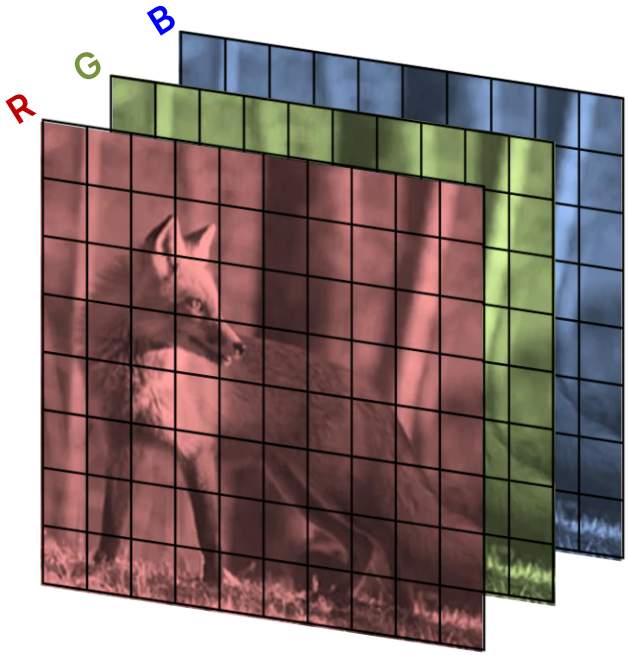


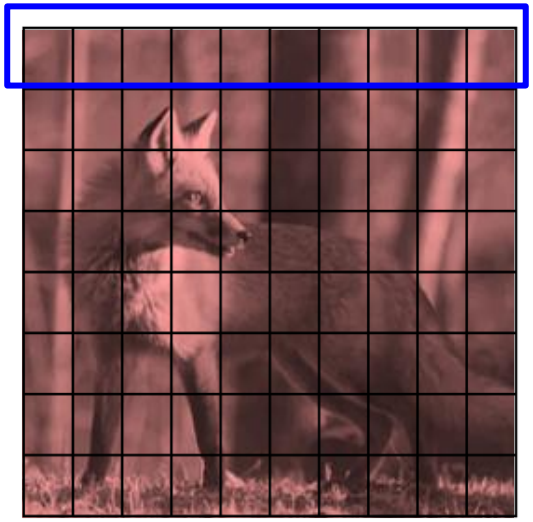
fig-6 matrix with color-range

Source : https://medium.com/swlh/understanding-the-mnist-and-building-classification-model-with-mnist-and-fashion-mnist-datasets-76b795da84b7

5. 차원별 텐서



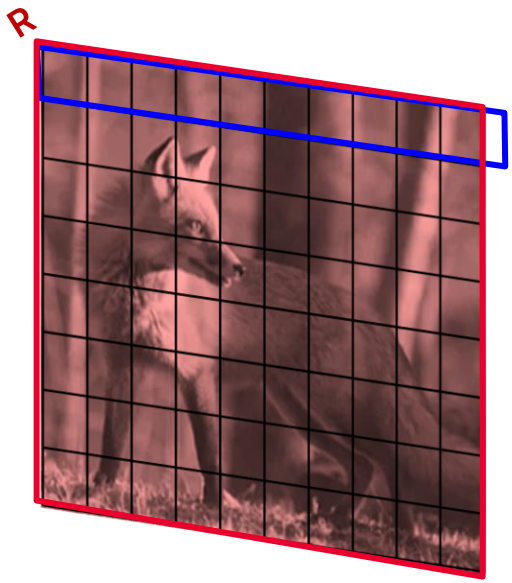




1D-Tensor
[R(1, 1), R(1, 2), ..., R(1, 640)]

가로 640개 = 가로 640px
Shape : (640)

5. 차원별 텐서

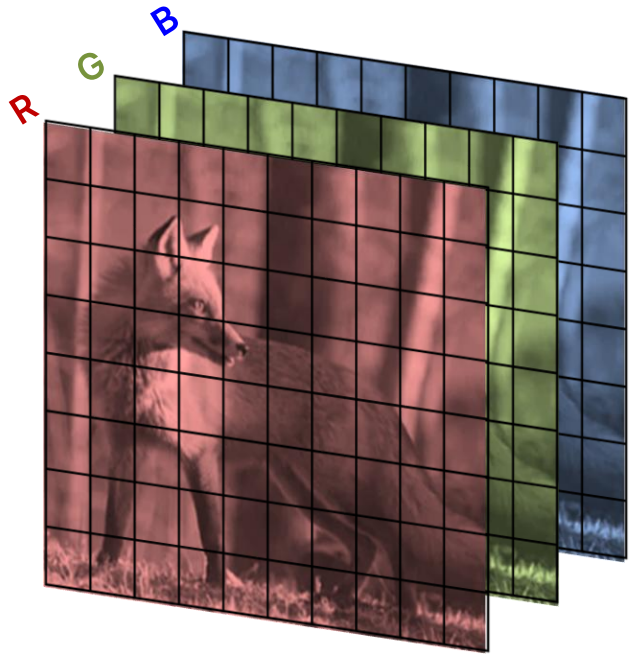


1D-Tensor
[R(1, 1), R(1, 2), ..., R(1, 640)]

2D-Tensor
[
  [R(1, 1), R(1, 2), ..., R(1, 640)],
  [R(2, 1), R(2, 2), ..., R(2, 640)],
  ...,
  [R(640, 1), R(640, 2), ..., R(640, 640)],
]

새로 640개 = 새로 640px
Shape : (640, 640)

5. 차원별 텐서



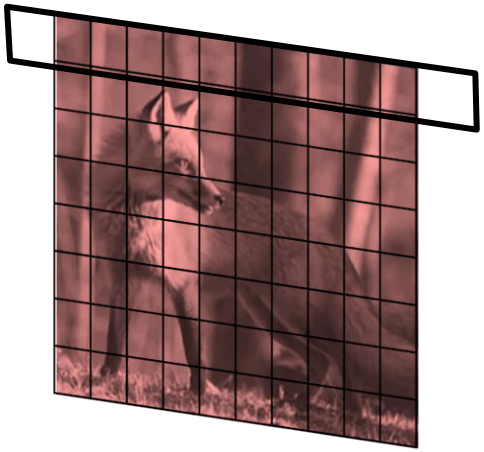
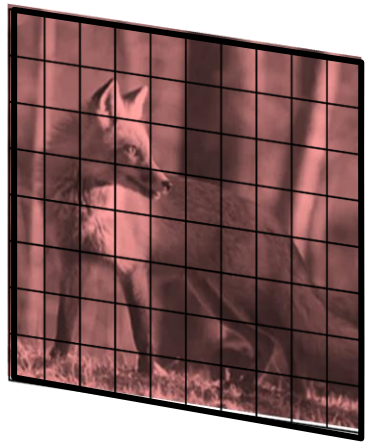
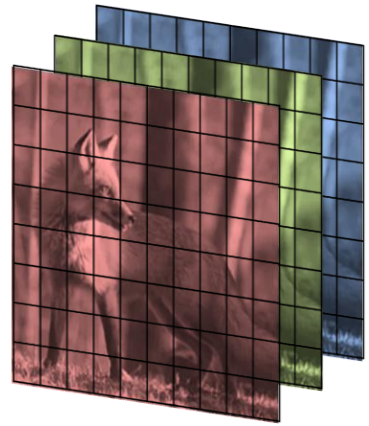
3D-Tensor

```
[
 [
   [R(1,1), G(1,1), B(1,1)],
   [R(1,2), G(1,2), B(1,2)], ...,
   [R(1,640), G(1,640), B(1,640)]
 ],
...,
 [
   [R(640,1), G(640,1), B(640,1)],
   [R(640,2), G(640,2), B(640,2)], ...,
   [R(640,640), G(640,640), B(640,640)]
 ]
]
```

채널 3개씩 묶어서
가로 640px, 새로 640px
Shape : (640, 640, 3)

6. Tensor Shape

- 데이터가 배열로 어떻게 구성되어 있는지를 나타내는 정보
- 각 차원이 얼마나 많은 요소를 포함하고 있는지를 차례대로 설명하는 방식

1D-Tensor	2D-Tensor	3D-Tensor
• **1차원의 길이** • **Shape : (길이)**	• **X-Y, 두 축의 크기(길이)** • **Shape : (가로 크기, 세로 크기)**	• **X-Y-Z, 3개 축의 크기(길이)** • **Shape : (가로 크기, 세로 크기, 채널수)**
		
• Shape : (640) • 길이가 640인 1차원 배열 • 텐서에는 640개의 값이 포함되어 있음	• Shape : (640, 640) • 640개의 행(row)과 640개의 열(column) • 텐서에는 640*640개의 픽셀값을 가지고 있음	• Shape : (640, 640, 3) • 640*640 크기의 이미지에서 픽셀이 3개(R, G, B)인 3차원 배열

[참고] Tensor Shape 쉽게 읽기



1. 가장 바깥쪽 대괄호부터 확인 = 첫번째 차원의 크기 = 2
2. 첫번째 차원의 그다음 차원의 크기 확인 = 두 번째 차원의 크기 = 3
3. 그 다음 차원의 크기 확인 = 세 번째 차원의 크기 = 1

➔ Shape : (2, 3, 1)

Example

[[A, B, C], [D, E, F]]

- 첫번째 차원의 크기 = 2
- 두번째 차원의 크기 = 3

➔ Shape : (2, 3)

[A, B, C, D]

- 첫번째 차원의 크기 = 4

➔ Shape : (4,)

TODAY QUIZ

1. 흑백 이미지 한 장을 2D-Tensor로 표현하고자 합니다.
   이미지의 가로와 세로 크기는 각각 128 픽셀 입니다.
   이 때 텐서의 차원과 shape은 어떻게 될까요?

   A. 1D-Tensor, Shape : (128)
   B. 2D-Tensor, Shape : (128, 128)
   C. 3D-Tensor, Shape : (128, 128, 1)
   D. 3D-Tensor, Shape : (128, 128, 3)

TODAY QUIZ

2. 가로와 세로가 각각 256 픽셀인 컬러 이미지를 다루고 있습니다.
   해당 이미지의 Tensor Shape은 무엇일까요?

A. Shape : (256)

B. Shape : (256, 256)

C. Shape : (256, 256, 3)

D. Shape : (3, 256, 256)

TODAY QUIZ

3. 아래의 Tensor Shape은?

```
[
  [
    [[1, 2], [2, 3], [3, 4]]
  ],
  [
    [[5, 6], [6, 7], [7, 8]]
  ]
[
```

TODAY QUIZ

4. 1D-Tensor [4, 7, 9, 1, 3, 8] 이 있습니다.
   해당 텐서를 (2, 3) 형태의 2D-Tensor로 변환하고자 합니다
   변환된 텐서는 다음 중 무엇인가요?

A. [[4, 7], [9, 1], [3, 8]]

B. [[4, 7, 9], [1, 3, 8]]

C. [[4, 9, 3], [7, 1, 8]]

D. 변환할 수 없음

TODAY QUIZ

5. 다음의 2D-Tesnor를
행 구조를 유지하면서 3D-Tensor의 (2, 1, 3) shape으로 변환하고자 합니다.
알맞게 변환된 것은 무엇인가요?
[[1, 2, 3],
[4, 5, 6]]

A. [[[1], [2], [3]], [[4], [5], [6]]]
B. [[[1, 2], [3, 4], [5, 6]]]
C. [[[1, 2, 3]], [[4, 5, 6]]]
D. 변환할 수 없음

TODAY QUIZ

6. Shape (2,3) 텐서 [[1, 2, 3], [4, 5, 6]]을 transpose하여 (3,2)로 만들었습니다. 변환된 텐서는 다음 중 무엇인가요?

A. [[1, 2], [3, 4], [5, 6]]

B. [[1, 4], [2, 5], [3, 6]]

C. [[6, 5], [4, 3], [2, 1]]

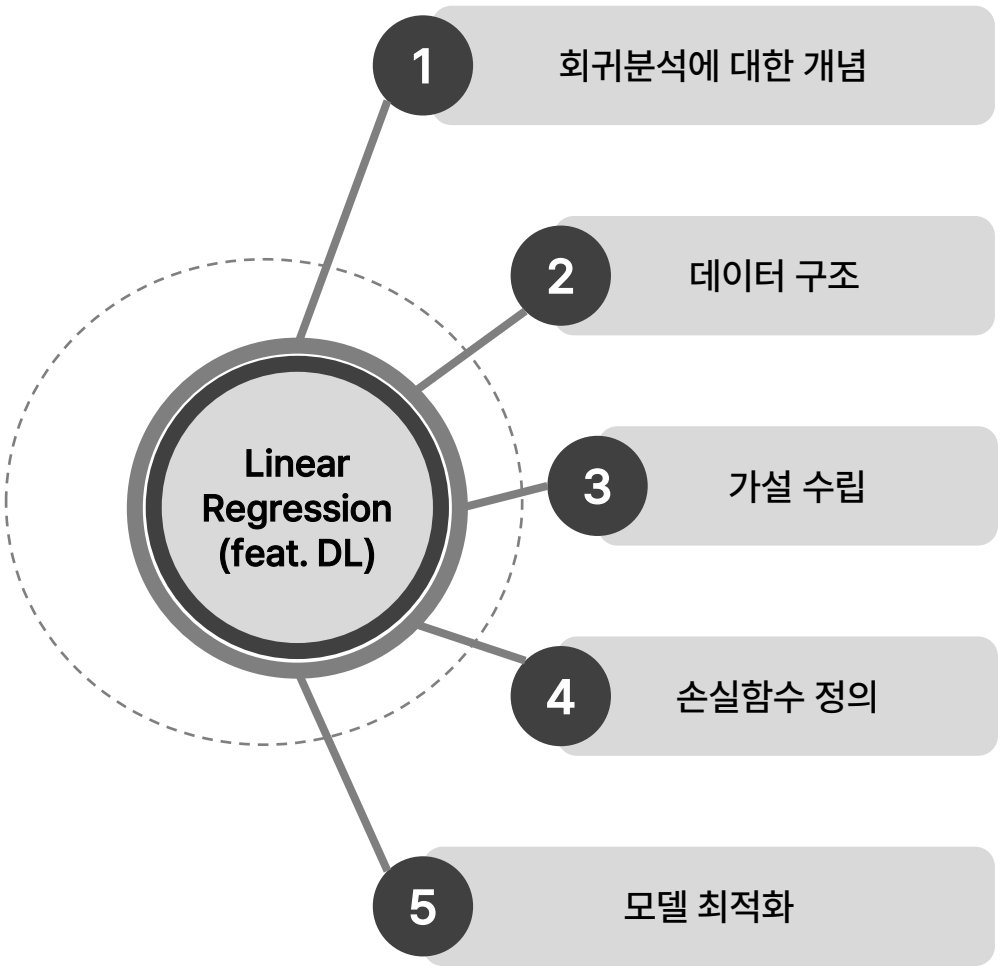
D. [[1, 2, 3], [4, 5, 6]]

Part-B. Deep Learning

SUMMARY

SUMMARY

Linear Regression 재해석

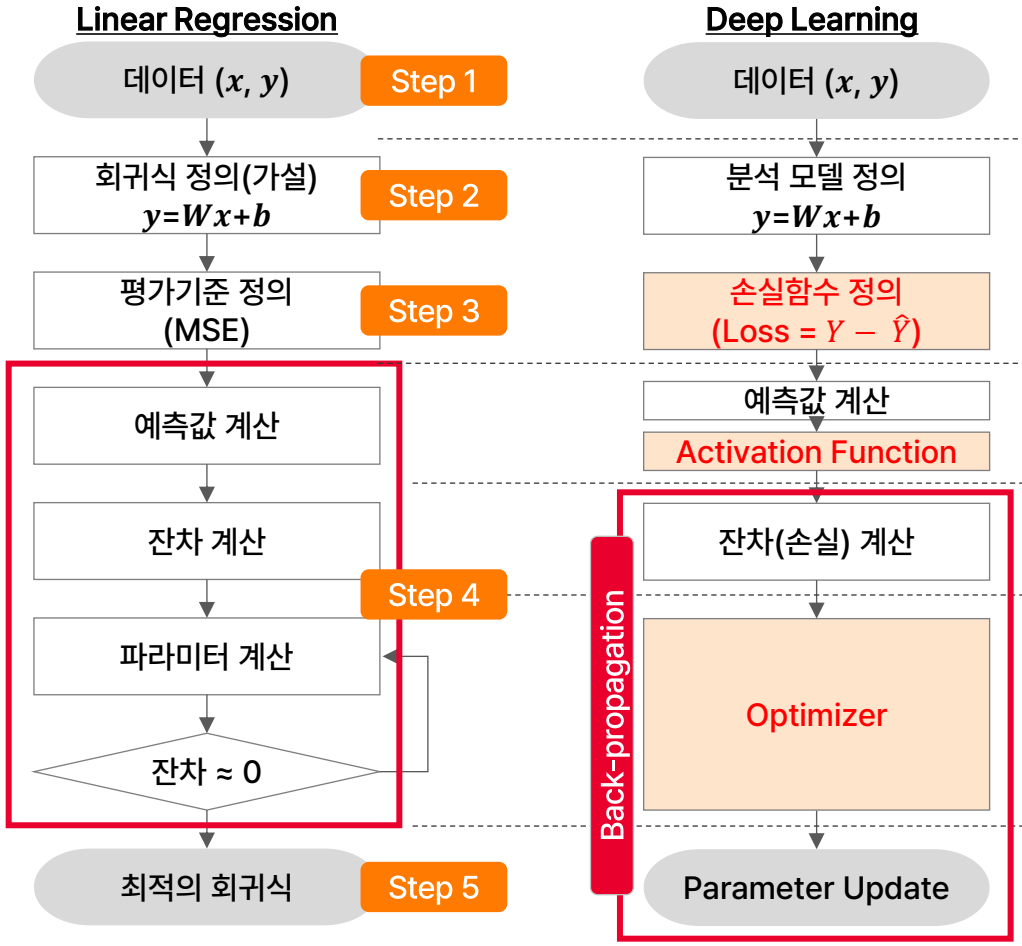


- **회귀분석에 대한 개념 이해**
  - 회귀 분석의 정의 및 활용 목적
  - 딥러닝 적용 위해 필요한 추가 절차
- **Data Definition, 데이터 구조에 대한 이해**
  - 딥러닝에 적합한 데이터 형태
  - Tensor Shape 이해
- **수립한 가설 기반으로 모델 개발**
  - 가설 수립 방법의 이해
  - 선형모델 또는 딥러닝 신경망 네트워크 아키텍처 구조 적용
- **손실함수 정의 및 계산**
  - 모델의 적합 여부를 평가하기 위한 기준의 손실 함수 정의/적용
  - 목적별 손실함수 이해
- **최적화 개념 및 방법**
  - Gradient Descent 개념 이해
  - 최적화 종류 및 종류별 장단점 이해/적용

SUMMARY

3. 회귀식을 구하는 과정

Statistics vs Deep Learning



- 데이터 수집/저장
- 학습 데이터셋 정의 (x →Feature, y →Target)

- **문제 정의 따라 Regression or Classification 정의**
  - 평가 기준 [ML] MSE (실제값 – 예측값), RMSE ($\sqrt{MSE}$)
    [DL] 손실함수, 비용함수 Cross Entropy *Classification*
  - ← **손실(오차)를 수치적으로 측정해 Backpropagation에 필요 정보 제공**

- **학습 모델 통해 예측값 계산**
  - 예측값에 Activation Function을 적용하여 **비선형성 추가** (ReLU)

- **잔차(오차, 손실, 비용함수) 계산**
  - **잔차를 줄이기 위해** Back-propagation **진행**

- **파라미터(W, b) 업데이트**
  - 파라미터를 업데이트 하는 규칙을 정의하고, 이를 통해 W와 b 값 조정
  - 오차가 최소(잔차 ≈ 0)가 되는 최적의 파라미터 계산 (**경사하강법**)
  - Back-propagation**은 손실이 가장 빠르게 줄어드는 방향으로, 최적화 알고리즘으로 가중치와 파라미터를 어떻게 조정할지 결정**

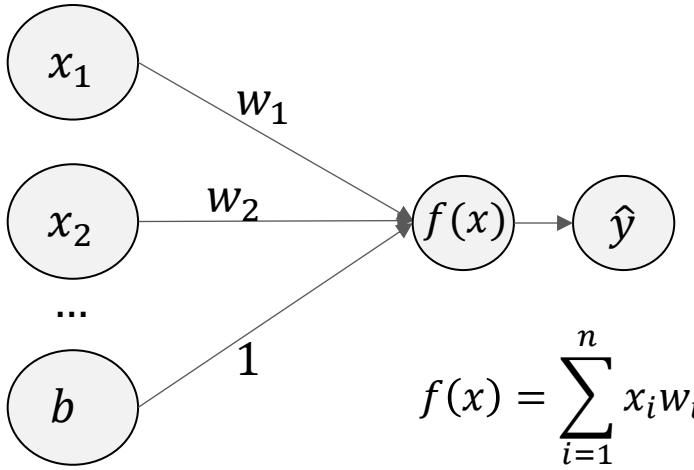
- **최적의 회귀식 추정**
  - 오차가 최소가 되는 최적의 파라미터로 최적의 회귀식 정의
  - **계산된 조정값을 적용하여 가중치 업데이트 (반복학습,** Epoch)

SUMMARY

3. 회귀식을 구하는 과정

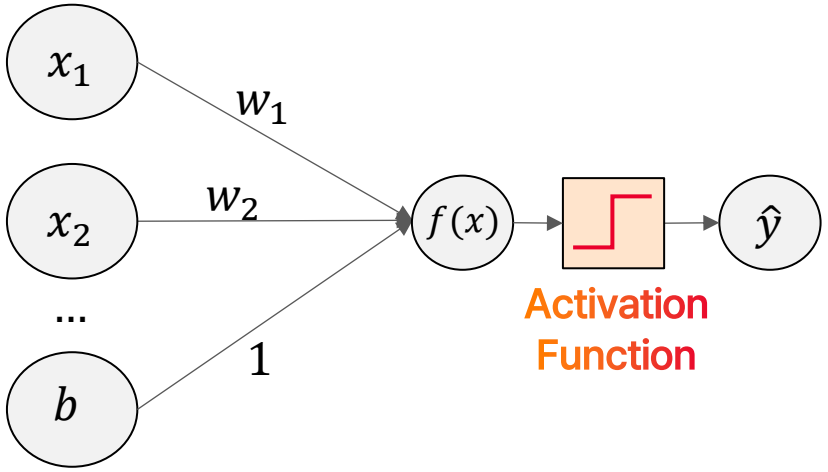
feat. Activation Function

Linear Regression



선형 회귀

Deep Learning



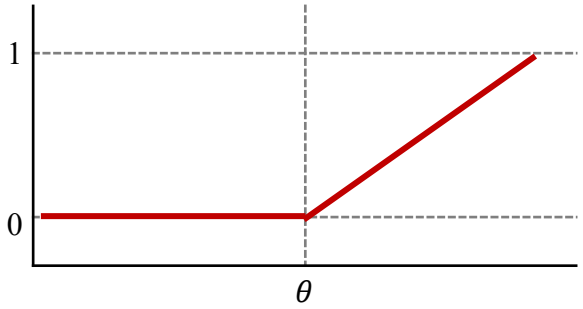
선형 회귀 + 활성함수

선형 회귀식을 비선형으로 만드는 활성함수를 가진 노드를 레이어로 쌓아 딥러닝 아키텍처로 구성

SUMMARY

4. Activation Function

ReLU – Rectified Linear Function



- 정의 : $ReLU(x) = \max(0, x)$
- 출력 : $(0, \infty)$
- 특징 : 입력이 양수인 경우 그대로 출력, 음수인 경우 0을 출력

→ 비선형성을 추가하면서도 계산이 간단하고 효율적이라 많이 사용됨

입력된 신호에서 중요한 부분은 그대로 전달,
필요 없는 음의 값은 0으로 만들어서
효율적으로 학습할 수 있도록

신경망이 다양한 특성과 패턴을 인식하고
정확하게 예측/분류 하는데 필수

예시: 사과와 오렌지 구분하기

1. 문제 설정:

- 사과와 오렌지를 구분하기 위해 우리는 크기(예: 지름)와 색상(예: 빨강, 주황 정도)을 특성으로 사용한다고 가정합시다. 이 특성들은 각 과일을 설명하는 데이터 포인트로 생각할 수 있습니다.

2. 데이터 입력과 ReLU의 적용:

- 딥러닝 모델에 크기와 색상을 입력하면, 입력값은 첫 번째 층의 뉴런에 전달됩니다. 각 뉴런은 입력값에 가중치를 곱하고 편향을 더한 후, ReLU 함수를 적용합니다.
- 예를 들어, 어떤 뉴런이 사과와 오렌지를 구분하려고 할 때, 사과일 가능성이 높은 특성 조합(크기가 크고 빨간색)이 들어오면 ReLU는 이 값을 그대로 유지합니다. 반면, 오렌지일 가능성이 높은 특성 조합(크기가 작고 주황색)이 들어오면 그 값 중 일부는 0으로 줄여 버립니다.

3. ReLU의 역할:

- ReLU는 입력된 값에서 의미 있는 양성 신호(사과에 더 가까운 특성)를 그대로 전달하면서 의미 없는 음성 신호(오렌지에 가까운 특성)를 걸러내는 역할을 합니다.
- 예를 들어, 사과와 오렌지를 구분하는 데 크기가 중요한 경우, 크기가 0보다 작은 값이나 비슷한 경우에 ReLU는 이를 0으로 처리하여 신경망이 구분을 쉽게 하도록 돕습니다.

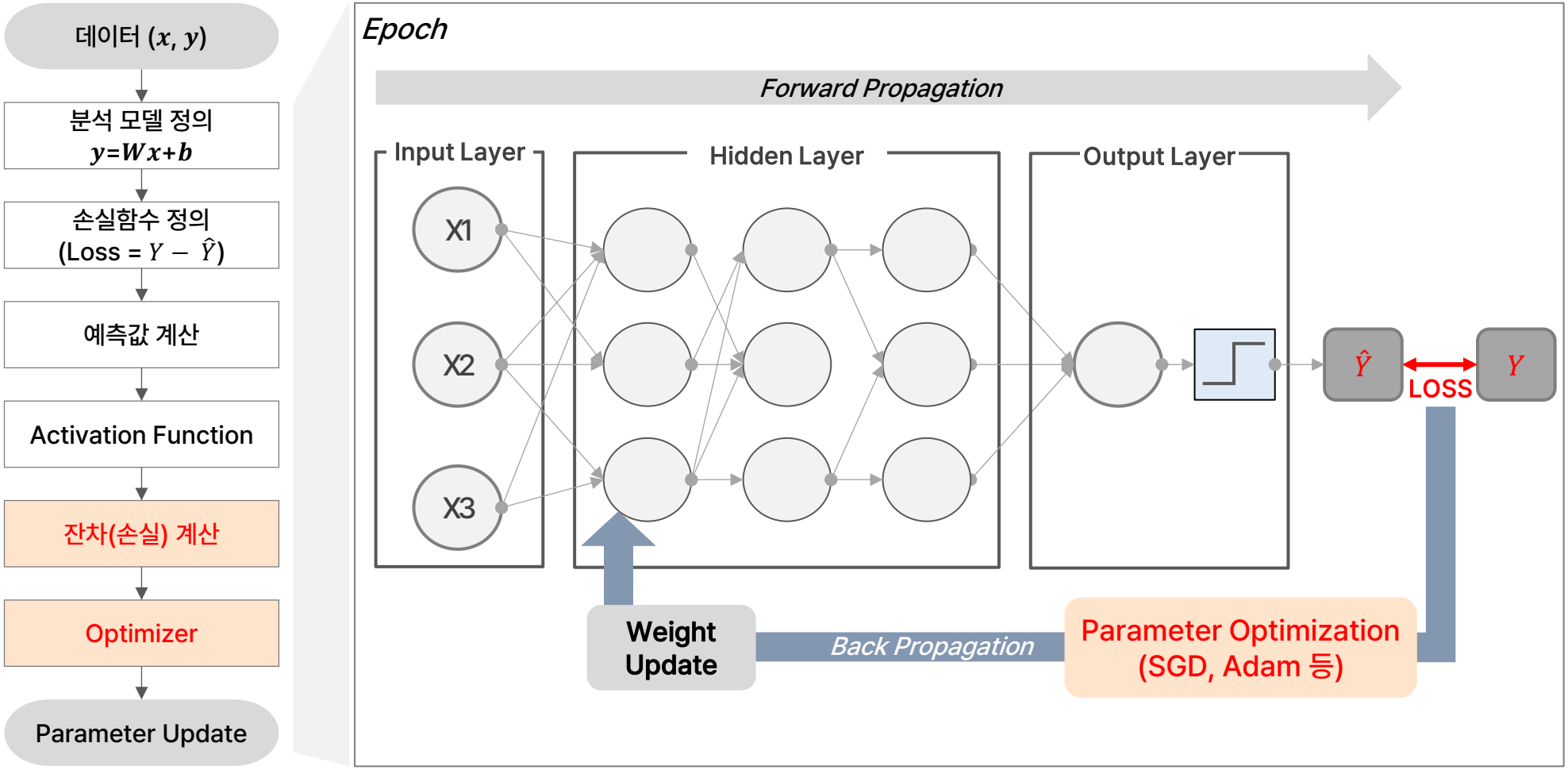
4. ReLU가 구분 경계에 미치는 영향:

- 신경망이 점차 훈련되면서, ReLU는 사과와 오렌지를 구분하는 결정 경계(비선형 경계)를 형성하는 데 기여합니다. 예를 들어, 뉴런이 사과를 대표하는 특성에 대해 큰 값을 유지하고, 오렌지에 대해선 0으로 만드는 방향으로 훈련된다면, 그 결과 사과와 오렌지를 구분하는 복잡한 비선형 경계가 형성됩니다.
- 이 경계는 단순한 직선이 아니라, ReLU의 비선형 특성 덕분에 더 복잡하고 정확하게 사과와 오렌지를 나눌 수 있는 모양으로 발전합니다.

SUMMARY

2. DL 최적화

DL 모델 최적화는 Loss Function을 최소화 하는 파라미터를 구하는 과정으로 경사하강법을 주로 사용함

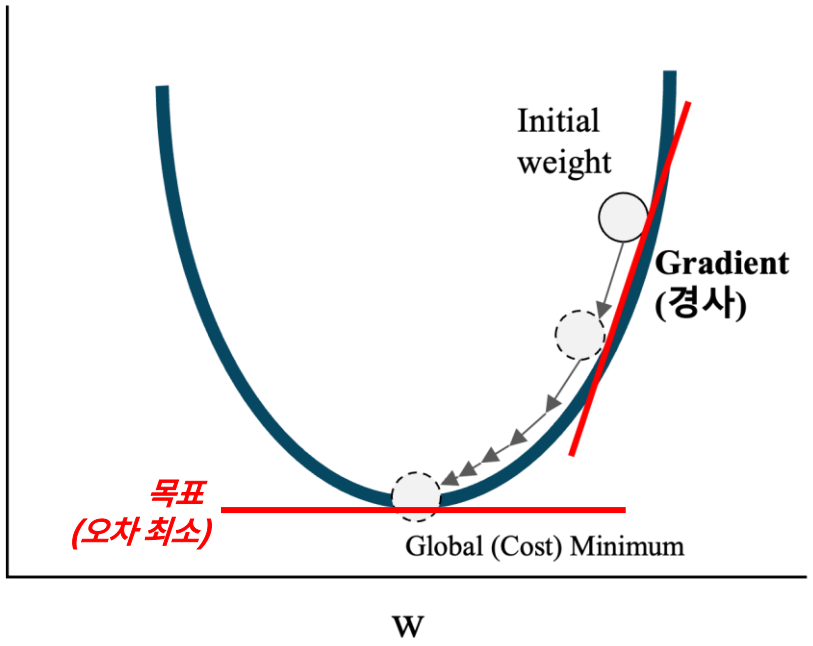


SUMMARY

4. Learning Rate

가중치가 얼마나 변할지, 업데이트 할지를 조정하는 값으로 목표 지점(Global Minimum)에 가까워질 수 있도록 최적화 필요

→ ***Hyper-parameter Tuning***



- 학습률, Learning Rate
  - 0과 1 사이의 값을 가지며, 사람이 직접 설정 (Hyper-parameter)
  - 작을수록 : 촘촘하게 파라미터를 변화시킴, 학습시간 오래 걸림
  - 클수록 : 손실함수 그래프 자체를 벗어나는 경우가 발생할 수 있음 (Over-shooting)
- 적정 학습률
  - 10-5승의 비율로 시작, Hyper-parameter Tuning 통해 최적화
  - 학습 초기에는 큰 스텝으로 빠르게 탐색하다가, 최적화 포인트 (전역 최소점, Global Minimum)와 가까워질수록 스텝을 작게 조정하여 정밀하게 탐색
- 기울기($\frac{\partial E}{\partial W_t}$)
  - 손실함수(E)의 가중치(W_t)에 대한 변화율
  - 현재 가중치에서 손실함수가 얼마나 빠르게 변하고 있는지 (학습)방향과 크기를 포함한 정보

경사하강법의 Learning Rate

새로운 가중치 W_t^* = 현재 가중치 W_t − 학습률 η * 기울기 $\frac{\partial E}{\partial W_t}$

현재 가중치(W_t)에서
학습률(η)에 손실함수의 가중치에 대한 기울기($\frac{\partial E}{\partial W_t}$) 를 사용해
새로운 가중치($* W_t$)를 계산

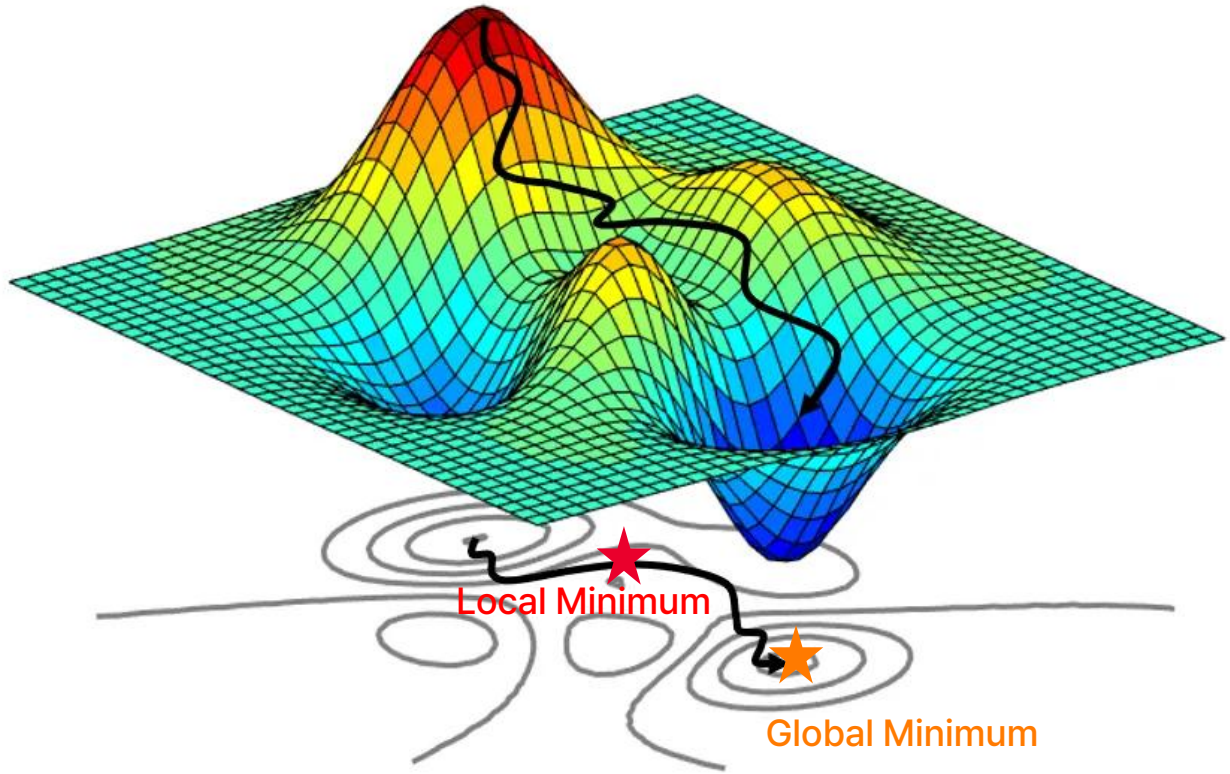
SUMMARY

[참고] Gradient Descent

가장 가파른 방향만 따라 내려가면 언젠가는 계곡에 도착하겠지만, 그 계곡이 진짜 우리가 도달해야 할 곳인가?

1. 목적 : 가장 낮은 곳(최소 손실값)을 찾는 것
2. 특징 : 지역 정보만 사용하여 산길 내려가기
3. 한계점
   - Local Minimum에 빠질 수 있음 (중간 계곡)
   - 경사면이 완만하면 느려짐
   - 출발 위치에 민감

   → 항상 정답으로 가는 것은 아님



Source : https://fabiorosato.com/blog/navigating-success-as-gradient-descent

SUMMARY

[참고] Gradient Descent

	Gradient Descent	Learning Rate	Gradient Vanishing	Gradient Clipping	Gradient Decay
대상	모델의 가중치 업데이트 방법	가중치 업데이트 속도	기울기(Gradient)	기울기(Gradient)	가중치(Weight)
목적	손실함수의 기울기(Gradient) 따라 가중치 업데이트	학습속도 조절, 기울기 크기에 따라 가중치 변화량 조절	신경망이 깊을 때 기울기가 사라져 학습이 멈추는 현상	기울기 폭발 방지, 학습 안전성 유지	과적합 방지, 모델의 일반화 성능 향상
작동 방식	손실함수의 기울기를 계산하여 가중치를 반대 방향으로 업데이트	학습률(Hyper-parameter) 따라 가중치 변화량 결정	역전파 과정에서 기울기가 점차 작아져 레이어에 도달하기 전에 사라짐	기울기가 설정된 임계값을 넘으면 그 값을 제한	손실함수에 가중치 크기에 대한 패널티 추가
장점	단순한 원리로 가중치 최적화	학습 속도를 조절하여 수렴을 가능하게 함 (Global Minimum)	학습이 진행되지 않는 현상 설명	학습의 안정성 보장, 기울기 폭발 방지	과적합 방지로 일반화 성능 향상
단점	Learning Rate 설정 따라 너무 느리게 수렴하거나 발산할 수 있음	잘못 설정되면 학습 실패 원인이 될 수 있음	기울기가 0에 가까워져 학습이 거의 이루어지지 않음	너무 작은 임계값 설정시 학습속도가 느려짐	정규화 정도가 너무 크면 모델 성능 저하

비유를 통한 설명 :

- Gradient Descent : 산을 내려가면서 최적의 지점을 찾는 과정 (손실 함수라는 산을 내려가면서 최소점을 찾기 위해 경사를 따라 이동함)
- Learning Rate : 산을 내려나는 등산 중에 걸음의 크기를 적절히 조절해 안정적으로 내려가는 것
  (너무 큰 걸음을 내딛으면 넘어지거나(발산) 목표를 지나칠 수 있고, 너무 작은 걸음을 내딛으면 시간이 오래 걸림)
- Gradient Vanishing : 짙은 안개로 길이 보이지 않는 상황처럼, 기울기가 사라져 학습이 잘 안되는 것
- Gradient Clipping : 가파른 경사를 줄로 제한해 넘어지지 않도록 하는 것
- Gradient Decay : 배낭의 무게를 줄여 가볍게 등산하는 것

SUMMARY

[참고] Adam Optimizer

산을 내려갈 때, 이전 경험을 기억하면서 더 똑똑하게 걷는 방법

- Gradient Descent는 단순하게 현재 경사(기울기)만 보고 내려감
- Adam은 내려가면서 생긴 피로도와 이전 경로의 방향성까지 고려해서 더 빠르게, 덜 흔들리며 내려감
  → 지형이 울퉁불퉁해도 효과적!

비유적 표현

- Gradient Descent는 등산객이 현재 위치에서 경사만 보고 움직이는 방식
- Adam은 등산객이 다음을 고려함:
  - 지금까지 어느 쪽으로 가고 있었나 : 방향 기억 – 기울기 평균(이전 방향들의 누적)
  - 이 방향이 얼마나 들쭉날쭉/가파른가 : 변동 크기 – 기울기 제곱의 평균

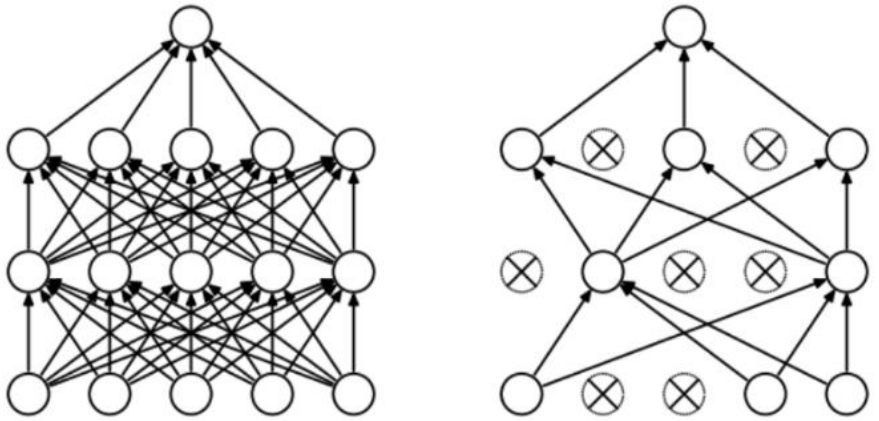
Adam Optimizer 흐름

① 현재 기울기 계산 : 지금 발 밑이 얼마나 가파른지 확인
② 1차 계산 (방향) : 지금까지 내려온 길을 종합해 속도/방향 정하기
③ 2차 계산 (변동) : 지금까지 내딛은 발걸음들이 발걸음들이 얼마나 들쭉날쭉했는지 누적으로 기록
④ 업데이트 : 넘어지지 않도록 균형 잡으면서 빠르게 내려가기

SUMMARY

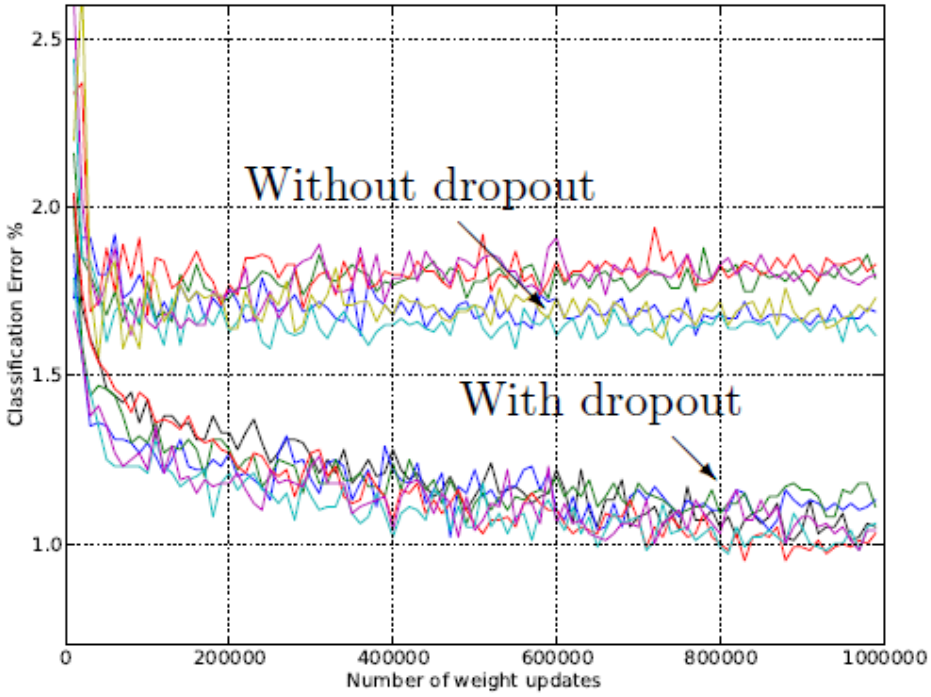
8. Dropout

Hidden Layer의 뉴런을 무작위로 골라 삭제하면서 학습 → 골고루 학습 효과



(a) Standard Neural Net

(b) After applying dropout.

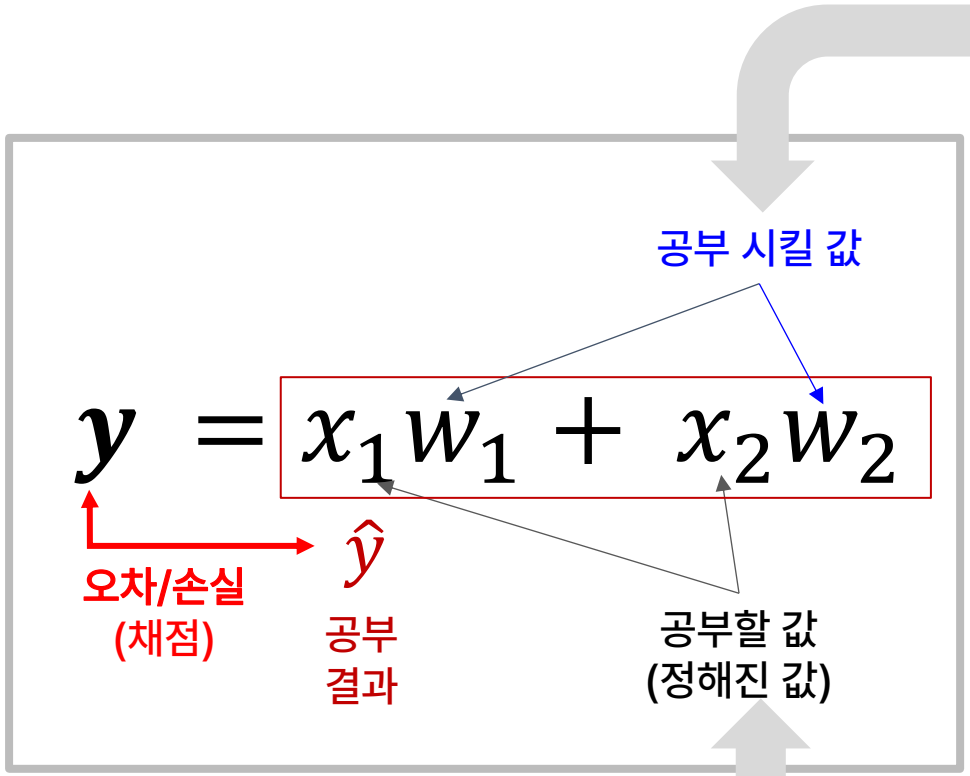


[주의]

- Training 단계 → 삭제할 뉴런을 무작위로 선택
- Testing 단계 → 학습한 모델의 결과를 확인해야 함으로
  모든 뉴런에 신호를 전달하여 확인

SUMMARY

Deep Learning Goal



무엇을, 언제까지 공부 시키지?

- 정답(Y)와 최대한 가까워지도록 **가중치**를 학습
  → **Gradient Descent** (Loss Function)
- **Overfitting** 발생하기 직전까지 학습
  → **Dropout** (학습 중단, early stopping)

학습 데이터가 큰데 어떻게 하지?

- **작은 셋으로 나누어** 학습 시키자
  → **Batch, Mini-batch** (데이터 잉여성 기반)
- **여러 번** 하면 데이터 특징을 더욱?!
  → **Epoch** (학습 데이터 소진 단위)

※ **잉여성**: input이 바뀌어도 output은 바뀌지 않는다

SUMMARY

2. Epoch, Batch, Mini-Batch

```
import torch
from torch.utils.data import DataLoader, TensorDataset

# (1) 더미 데이터셋 생성 (총 20개의 샘플, 각 샘플은 2개의 피처)
X = torch.arange(1, 41).reshape(-1, 2).float()  # shape: (20, 2)
y = torch.arange(1, 21).float().reshape(-1, 1)   # shape: (20, 1)

# (2) PyTorch Dataset 구성
dataset = TensorDataset(X, y)

# (3) DataLoader로 Mini-Batch 구성 (batch_size=5이면 mini-batch 4개)
batch_size = 5
loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

# (4) Epoch 설정
num_epochs = 3

# (5) 학습 루프 시작
for epoch in range(num_epochs):
    print(f"\n Epoch {epoch+1}/{num_epochs}")

    for batch_idx, (features, targets) in enumerate(loader):
        print(f"\n Mini-Batch {batch_idx+1}")
        print(f" - Features:\n{features}")
        print(f" - Targets:\n{targets}")
```

- **Epoch**
  - 전체 데이터셋을 한 번 학습
    → 파라미터를 한 번씩 업데이트하는 과정
  - (e.g.) 20개 데이터 : 1 epoch에서 20개 모두 사용
    num_epcohs = 3 : 위 과정을 3번 반복

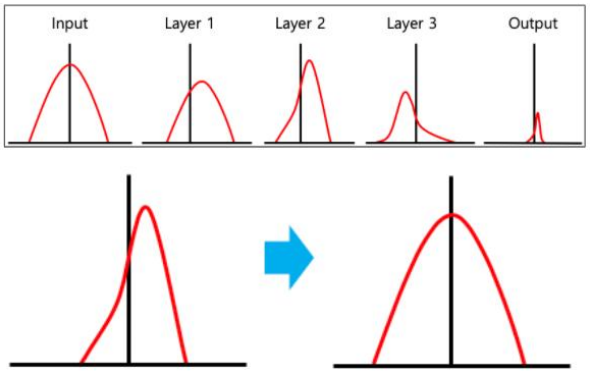
- **Batch**
  - 학습 시 한번에 처리하는 데이터 묶음
  - (e.g.) 총 20개 데이터셋을 한번에 모두 사용
    : batch_size = 20 → batch_size = 1

- **Mini-Batch**
  - Batch 중에서, batch_size가 전체 데이터보다 작은 경우
  - (e.g.) 총 20개 데이터셋 & batch_size = 5 :
    1 epoch 당, 4개의 mini-batch로 학습

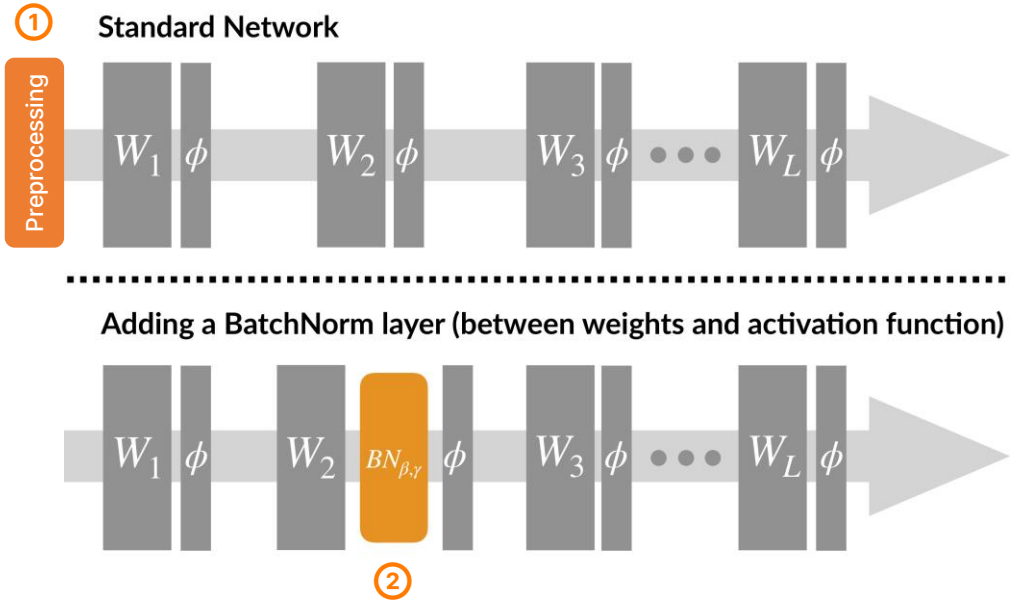
➔ 메모리 절약, 학습 속도 향상, gradient 안정성 확보

SUMMARY

3. Batch Normalization



- BN은 각 층의 입력을 정규화해 학습을 빠르고 안정적으로 만듦
- 손실 지형을 매끄럽게 해 기울기를 안정시키는 것이 핵심
- 그 결과 기울기 안정화(Vanishing 완화) · 더 큰 학습률 사용 가능 · 빠르고 안정적 수렴



- 기대효과:
  - Gradient 흐름 개선 : 입력 분포가 안정되면서 Vanishing 완화
  - 학습 속도 향상 : 더 큰 학습률(learning rate) 사용 가능
  - 학습 안정성 확보 : 각 층의 입력 분포가 일정하게 유지
  - Regularization 효과 : 과적합 방지 효과



[Optimization Landscape, 학습 공간]

BN은 공간을 매끄럽게 처리해 주어 경사가 지나치게 요동치거나 평평해지는 문제를 줄일 수 있음. 덕분에 gradient가 안정적으로 수렴하고, 더 큰 학습률에서도 학습이 수월하게 진행될 수 있음 → 모델은 빠르고 안정적으로 수렴할 수 있음

Source : https://gradientscience.org/batchnorm/ (modified), https://velog.io/@choiking10/Batch-Normalization에-대해서-알아보자



수고하셨습니다.

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX